

Assembly Part 2: de Bruijn Graphs Metagenomics.

Slides adapted from Fin Maguire
Who adapted most slides from Ben Langmead's Teaching Materials
(www.langmead-lab.org/teaching.html)

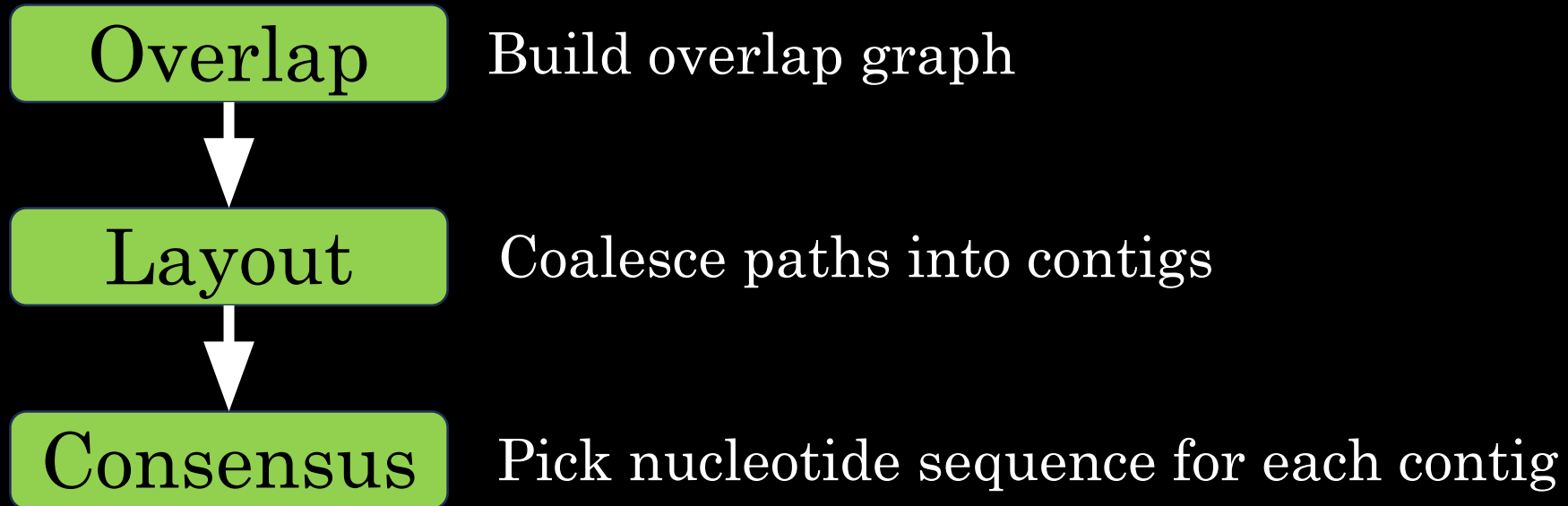
Various Course Things

- **Assignment #2 (Assembly): Due next Wednesday**
 - You should have a CSID (let us know ASAP if you can't find it)
 - A couple of steps take time, so verify that everything works and start this week
- **Paper report:**
 - Please try and share your ideas with me by end of next week (or sooner) so I can give feedback before you get locked into a challenging paper choice!

Overview

- Shortest Common Superstring
- de Bruijn graphs (DBGs) as an alternative to overlap-layout-consensus – welcome back k -mers
- How to error correct / reconcile DBGs
- Metagenomics: sequencing entire communities of organisms

Reminder: OLC and Its Limitations



- Building overlap graph is slow. $O(N + a)$, $O(N^2)$
- Overlap graph is **huge**: one node per read, # edges can grow superlinearly with # reads
- Sequencing datasets are ~ 100 s of millions or billions of reads

Another Method: Shortest Common Superstring (SCS)

- We have a bunch of reads:

ABA ABB AAA AAB BBB BBA BAB BAA

- We could do OLC as before, but a (simpler?) way is to find the shortest reconstruction that contains all input sequences:

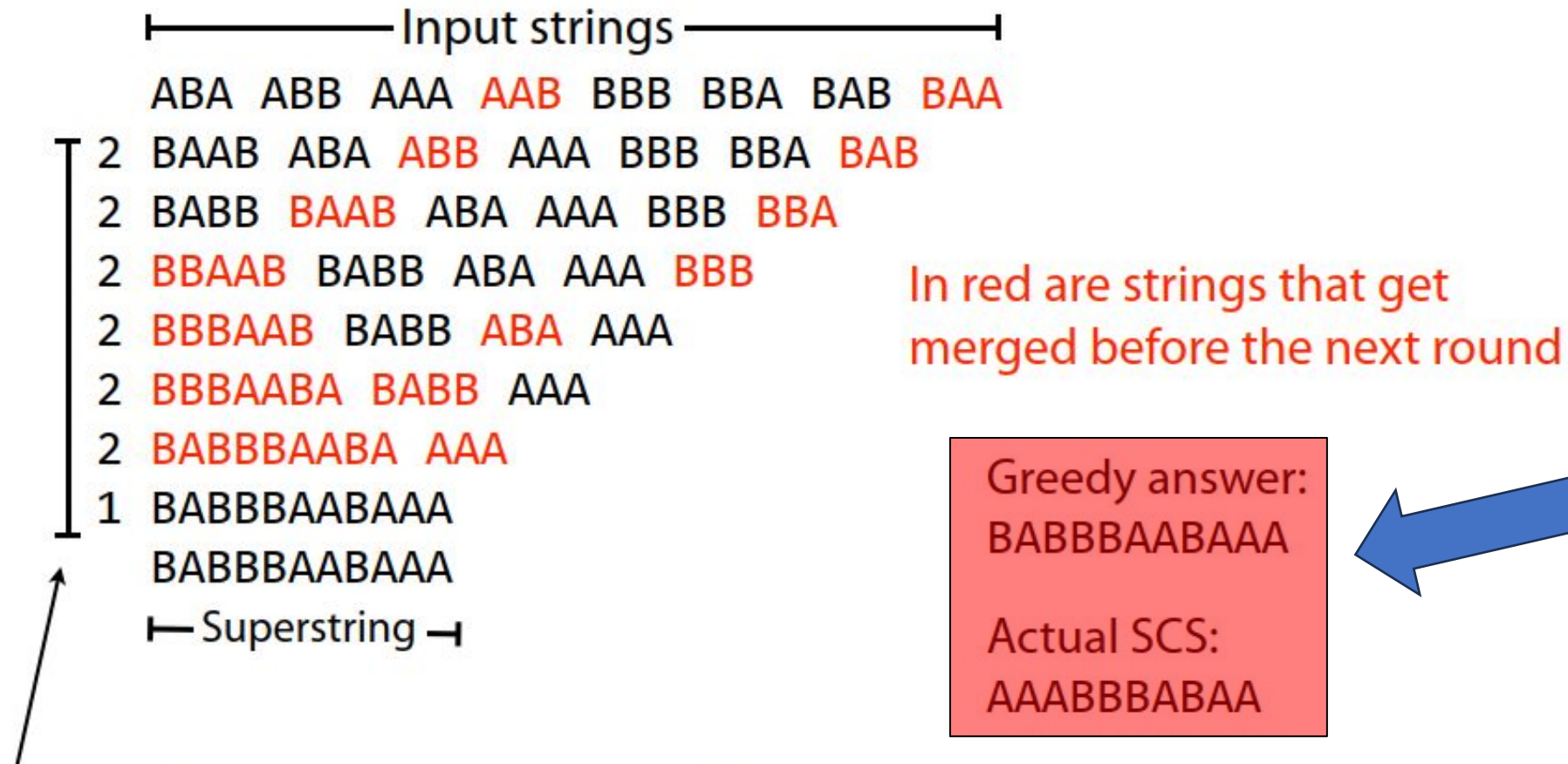
AAABBBABAA

- This is a **superstring**, and the **shortest** one is the simplest way to represent all input reads

Two Problems

- Optimal SCS is inefficient (the best-known solutions scale **exponentially** with the size of the input)
- So we can use a greedy approach instead, choosing the solution that seems best at each given point (Greedy-SCS):

Greedy-SCS



Rounds of merging, one merge per line.

Number in first column = length of overlap merged before that round.

So We Kinda Solved the Complexity Problem, BUT...

String = "a_long_long_long_time"

$k = 6, l = 3$

```
ng_lon _long_ a_long long_l ong_ti ong_lo long_t g_long g_time ng_tim
5 ng_time ng_lon _long_ a_long long_l ong_ti ong_lo long_t g_long
5 ng_time g_long_ ng_lon a_long long_l ong_ti ong_lo long_t
5 ng_time long_ti g_long_ ng_lon a_long long_l ong_lo
5 ng_time ong_lon long_ti g_long_ a_long long_l
5 ong_lon long_time g_long_ a_long long_l
5 long_lon long_time g_long_ a_long
5 long_lon g_long_time a_long
5 long_long_time a_long
4 a_long_long_time
a_long_long_time
```



We've lost a repeat, because no read tells us that there should be **three occurrences** of 'long'

So

- Shortest common substring: impractical, can oversimplify the problem
- Graph-based approaches will handle this better in general (as we'll see in a few minutes)
- Before we move on, an interesting bridge from OLC to what's coming next...

Whole-Genome Random Sequencing and Assembly of *Haemophilus influenzae* Rd

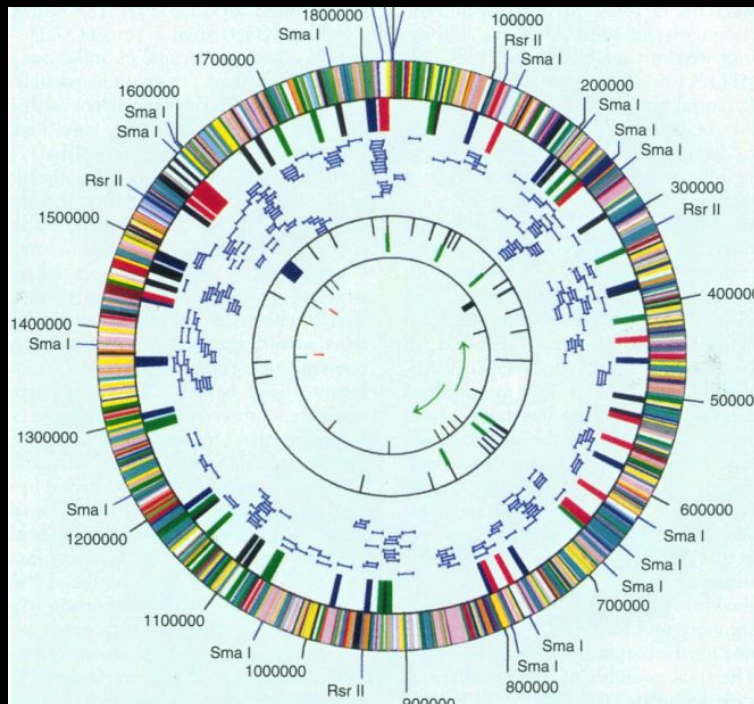
Robert D. Fleischmann, Mark D. Adams, Owen White, Rebecca A. Clayton, Ewen F. Kirkness, Anthony R. Kerlavage, Carol J. Bult, Jean-Francois Tomb, Brian A. Dougherty, Joseph M. Merrick, Keith McKenney, Granger Sutton, Will FitzHugh, Chris Fields,* Jeannine D. Gocayne, John Scott, Robert Shirley, Li-Ing Liu, Anna Glodek, Jenny M. Kelley, Janice F. Weidman, Cheryl A. Phillips Tracy Spriggs, Eva Hedblom, Matthew D. Cotton, Teresa R. Utterback, Michael C. Hanna, David T. Nguyen, Deborah M. Saudek, Rhonda C. Brandon, Leah D. Fine, Janice L. Fritchman, Joyce L. Fuhrmann, N. S. M. Geoghagen, Cheryl L. Gnehm, Lisa A. McDonald, Keith V. Small, Claire M. Fraser, Hamilton O. Smith, J. Craig Venter†

“The only complete genome sequence from a free-living organism” *Science* (1995)

1,830,137 base pairs. Shotgun sequence with gap filling

Sequenced using the TIGR Assembler (also published in 1995), which does OLC...

...BUT starts with a k-mer filter. Compare reads *only if they share at least one 10-mer*



Let's Take That k -mer Idea Several Steps Further...

de Bruijn Graphs

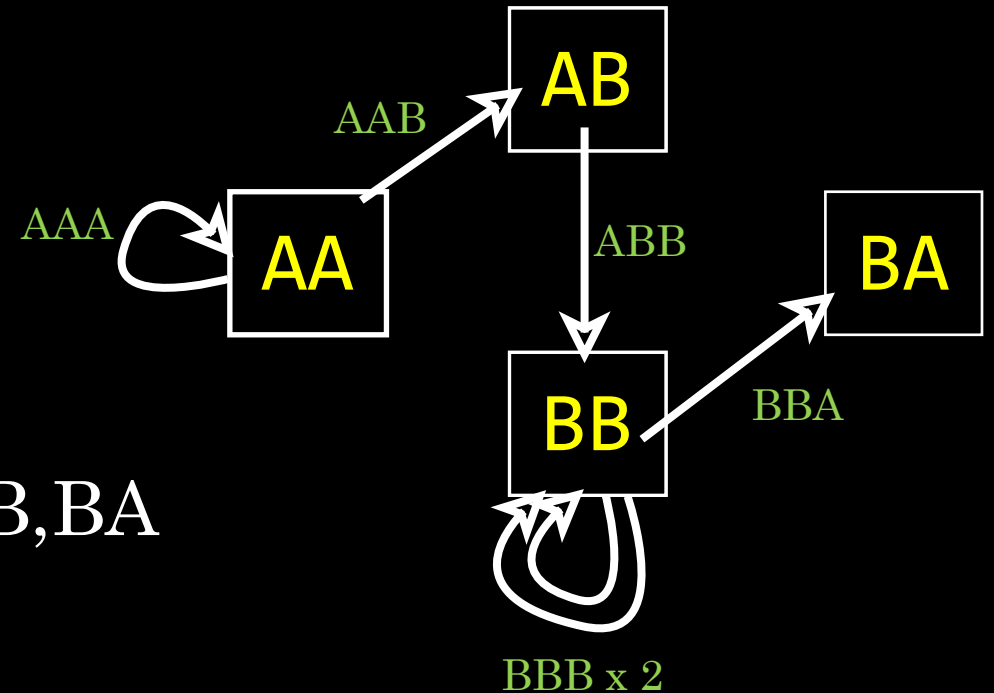
- Decompose the genome (reads) into a series of overlapping k -mers
- Nodes represent all distinct $(k - 1)$ – mers, **not individual reads**
- Edges represent k -mers that connect observed nodes
- Nomenclature mess: edge-centric vs node-centric *de Bruijn graphs*

A Simple *de Bruijn* Graph

Genome: **AAABBBBA**

Overlapping k -mers (edges):
AAA, AAB, ABB, BBB, BBB, BBA

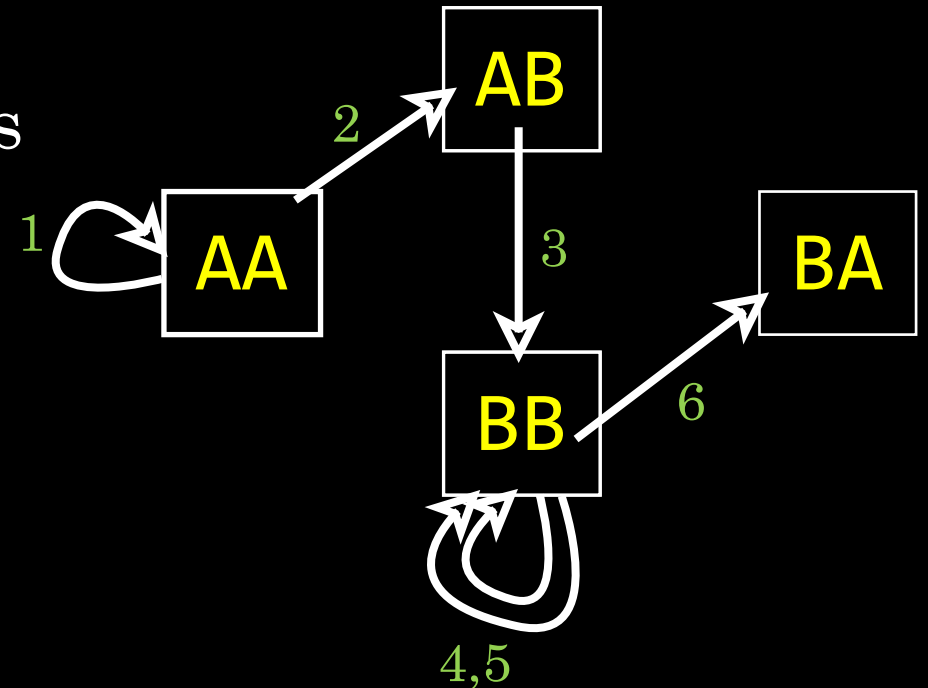
$(k - 1)$ -mers (nodes):
AA, AA, AA, AB, AB, BB, BB, BB, BB, BB, BA



A Simple *de Bruijn* Graph

Walk crossing each edge exactly once gives a reconstruction of the genome. This is an **Eulerian walk**

1 AAABBBBA
2
3
4
5
6

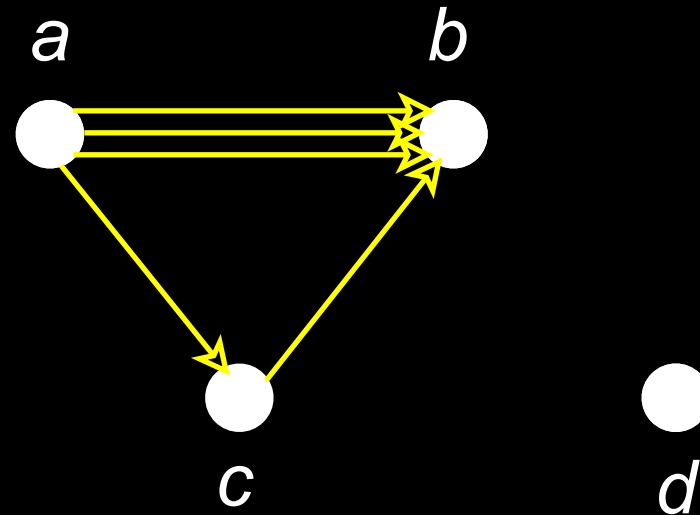


de Bruijn Graphs are Directed Multigraphs

A multigraph can have **multiple edges** between the same pair of nodes

Node **indegree** = # of incoming edges

Node **outdegree** = # of outgoing edges



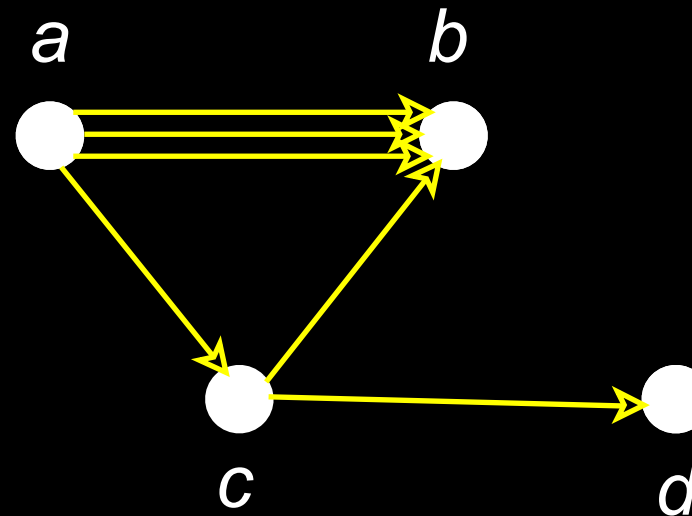
Vertices (V) = { a, b, c, d }

Edges (E) = { (a, b), (a, b), (a, b), (a, c), (c, b) }

Repeated

de Finitions #1

A directed graph is **(weakly) connected** if every node can be reached from every other node when we ignore direction



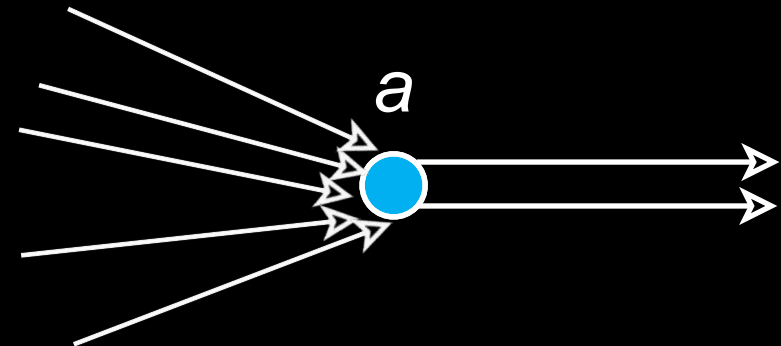
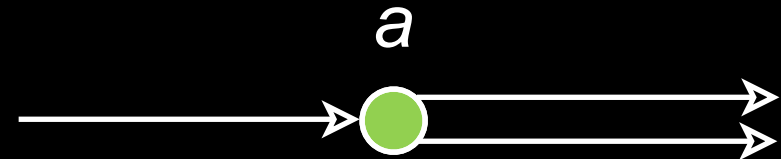
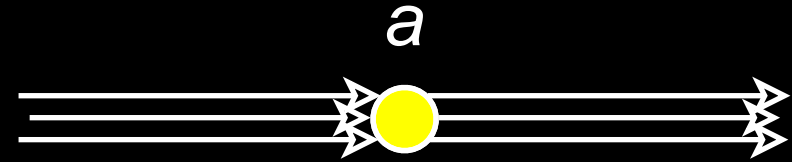
Unconnected

Connected

de Finitions #2

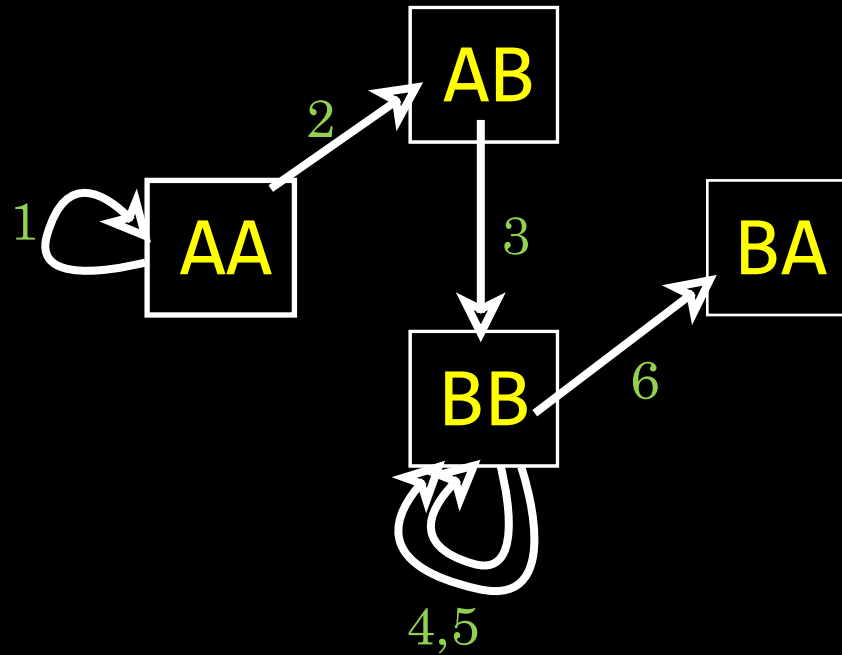
Nodes are:

- **Balanced** if $\text{indegree} = \text{outdegree}$
- **Semi-balanced** if indegree and outdegree differ by no more than 1
- **Imbalanced** if they differ by > 1



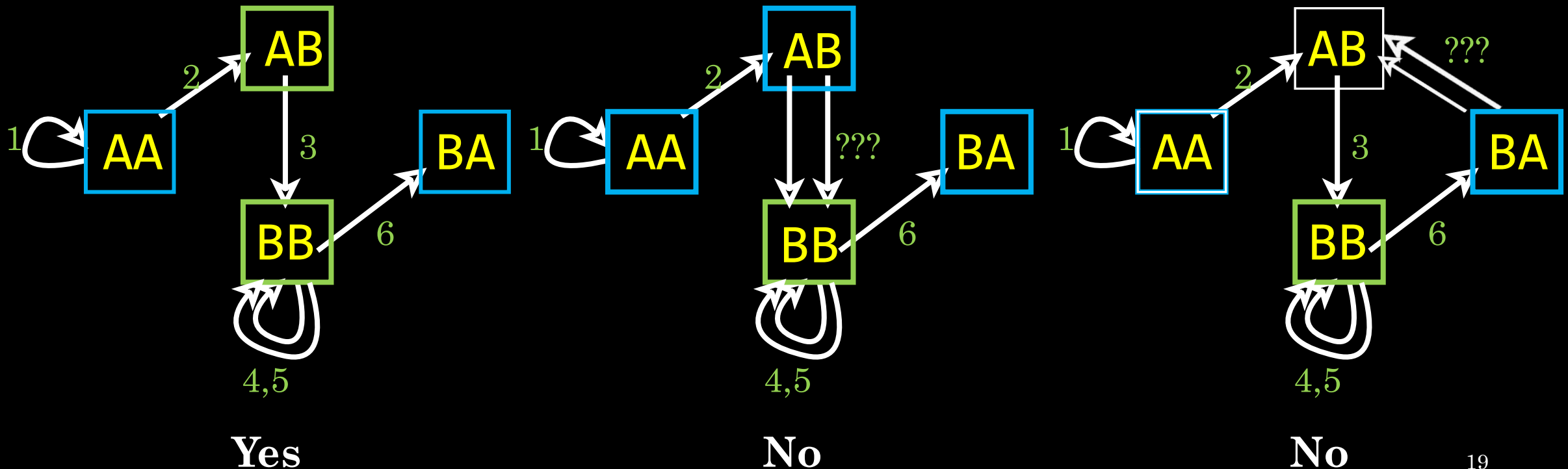
de Finitions #3

Eulerian walk visits each edge exactly once

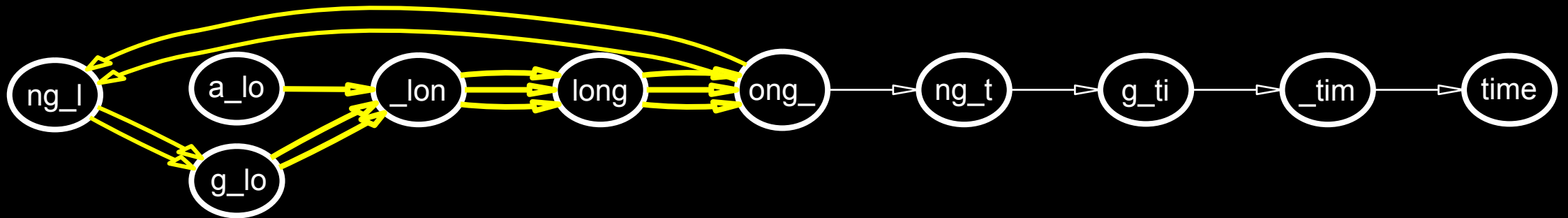


de Finitions #3

But graphs only have an Eulerian walk if they have at most two **semi-balanced nodes**, and all others are **balanced**.

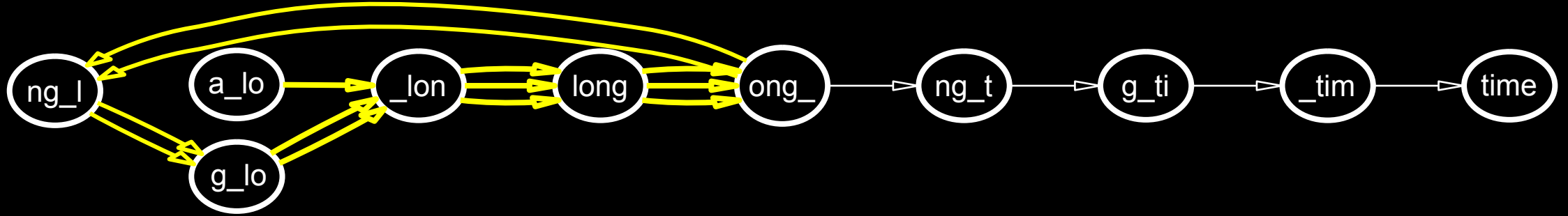


```
['a_lo', '_lon', 'long', 'ong_', 'ng_l', 'g_lo',  
 '_lon', 'long', 'ong_', 'ng_l', 'g_lo', '_lon',  
 'long', 'ong_', 'ng_t', 'g_ti', '_tim', 'time']
```



Eulerian walk correctly resolves the longer repeat

The Eulerian Walk Doesn't Solve All Our Problems Though...

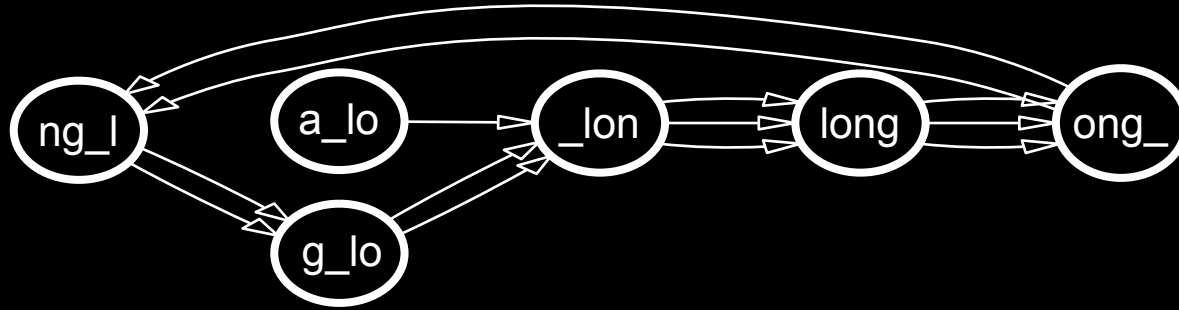


In our beautiful example:

- Coverage is uniform
- Repeats are contiguous
- No sequencing errors

Missing Reads

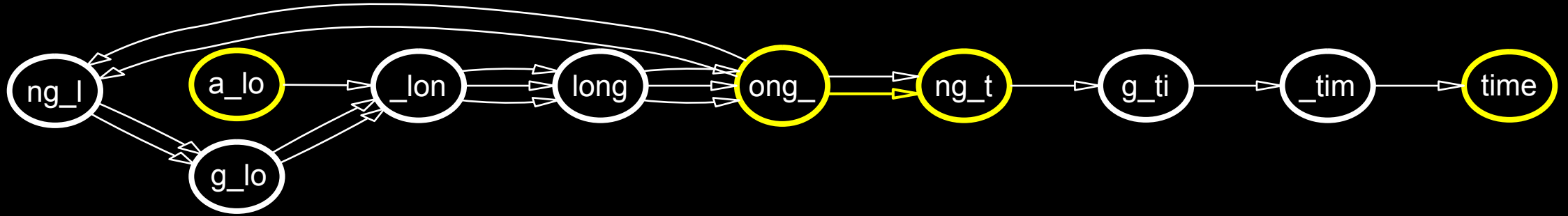
~~ng_t~~



Graph is **disconnected**

Imbalanced Reads

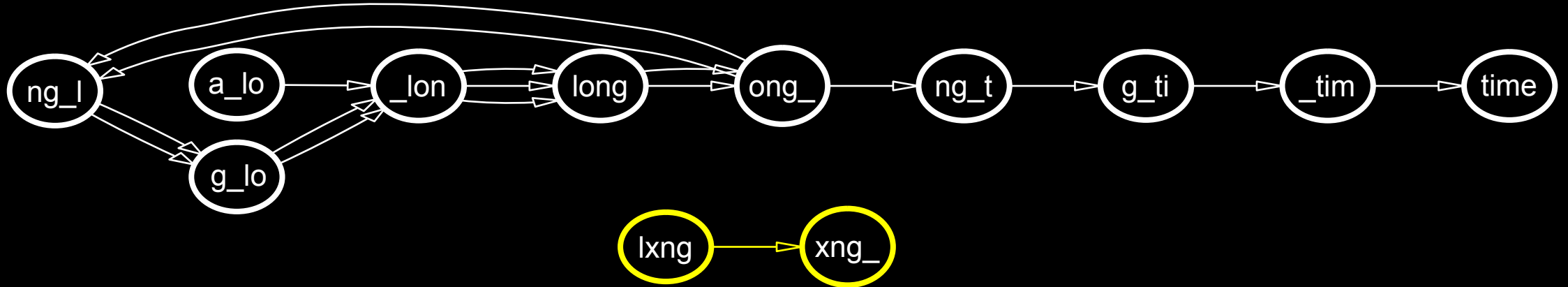
Add an extra 'ong_t':



Four semi-balanced nodes (non-Eulerian!)

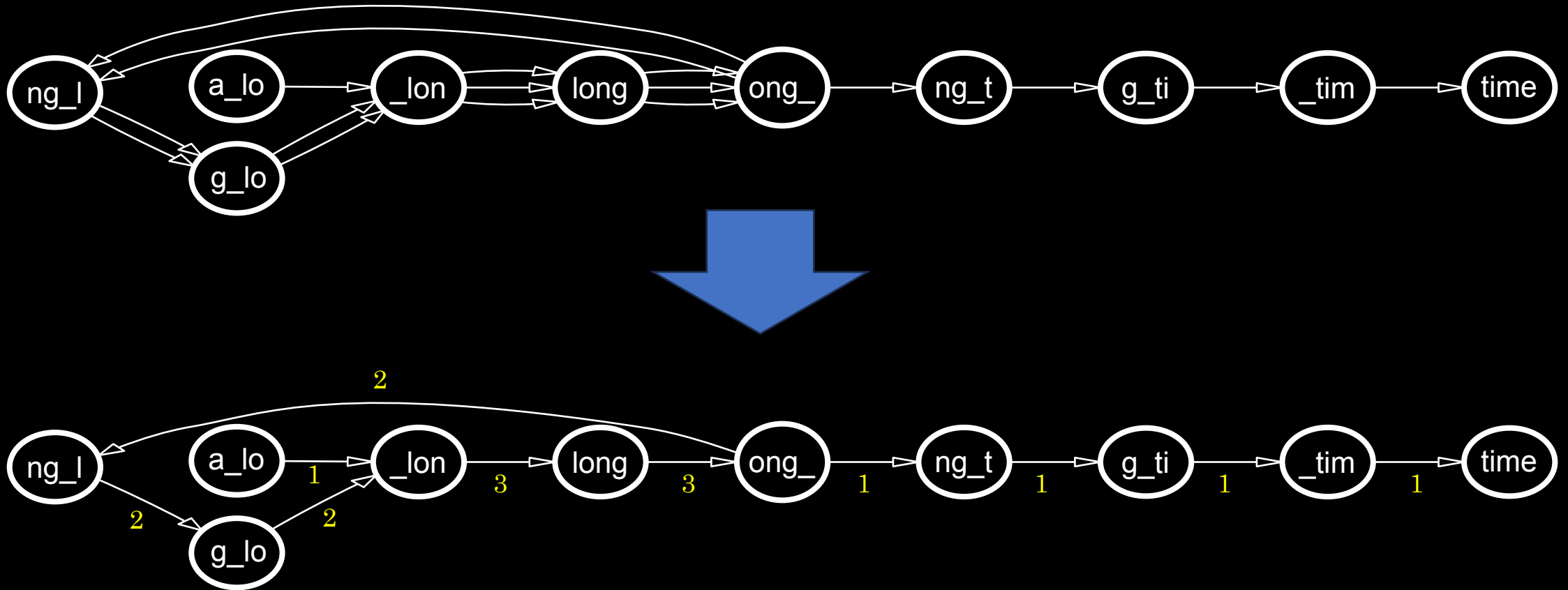
Sequencing Errors

One read is 'lxng_' instead of 'long_'



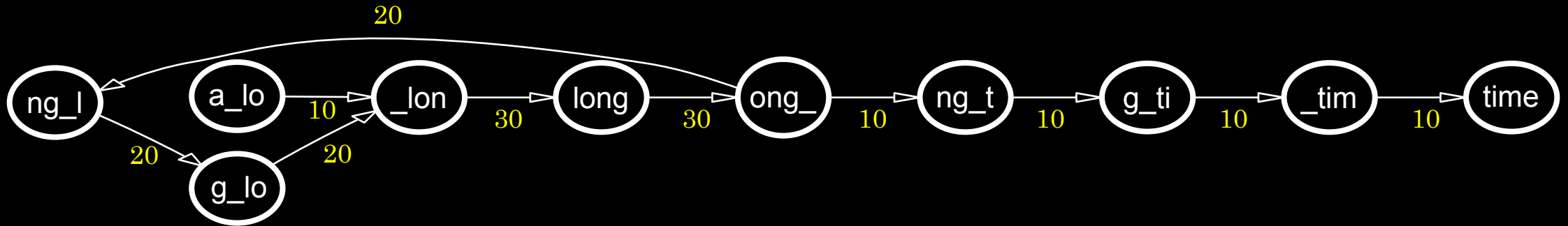
Making de Bruijn Graphs Practical

Weighted Graphs

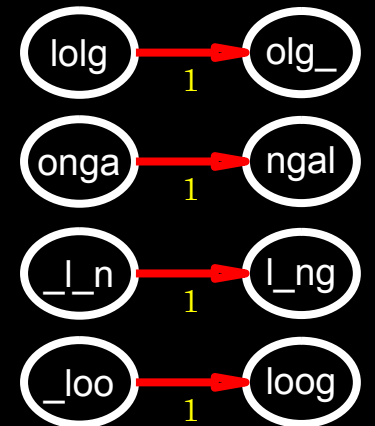
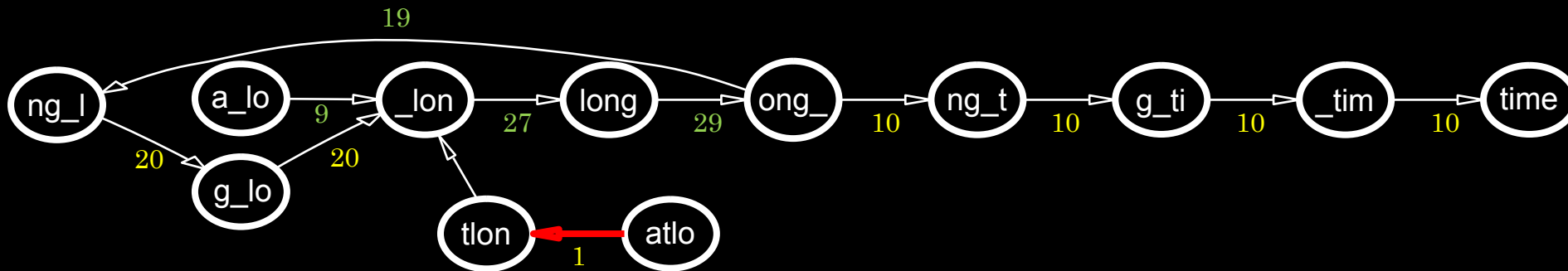


Error Correction

(Larger weights to make the problem a bit more interesting)

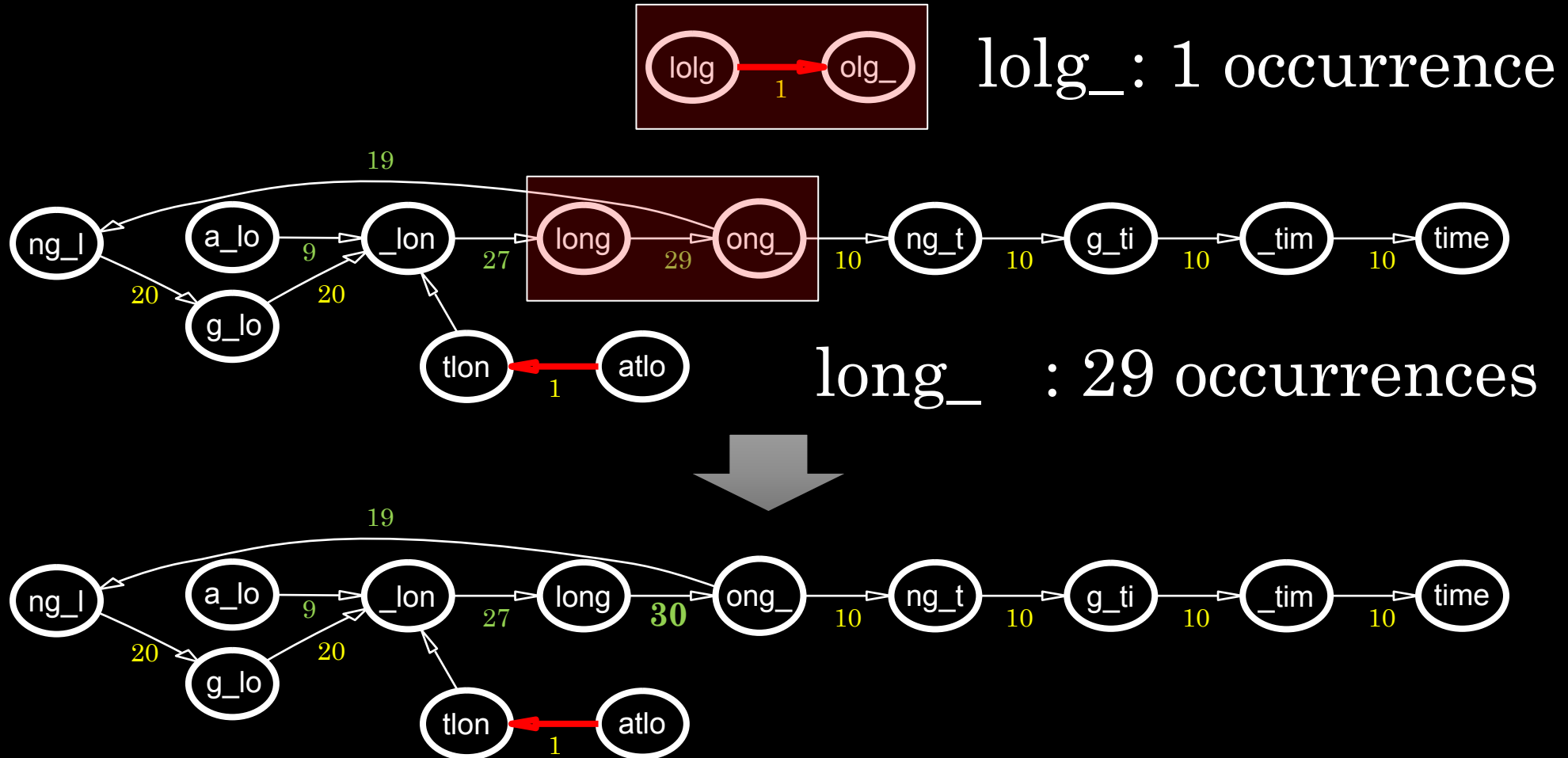


Introducing a 1% error rate creates **6 mutations**
The new nodes all have weight 1



Error Correction

Examine rare k -mers and see if any small variants of the k -mer are more common

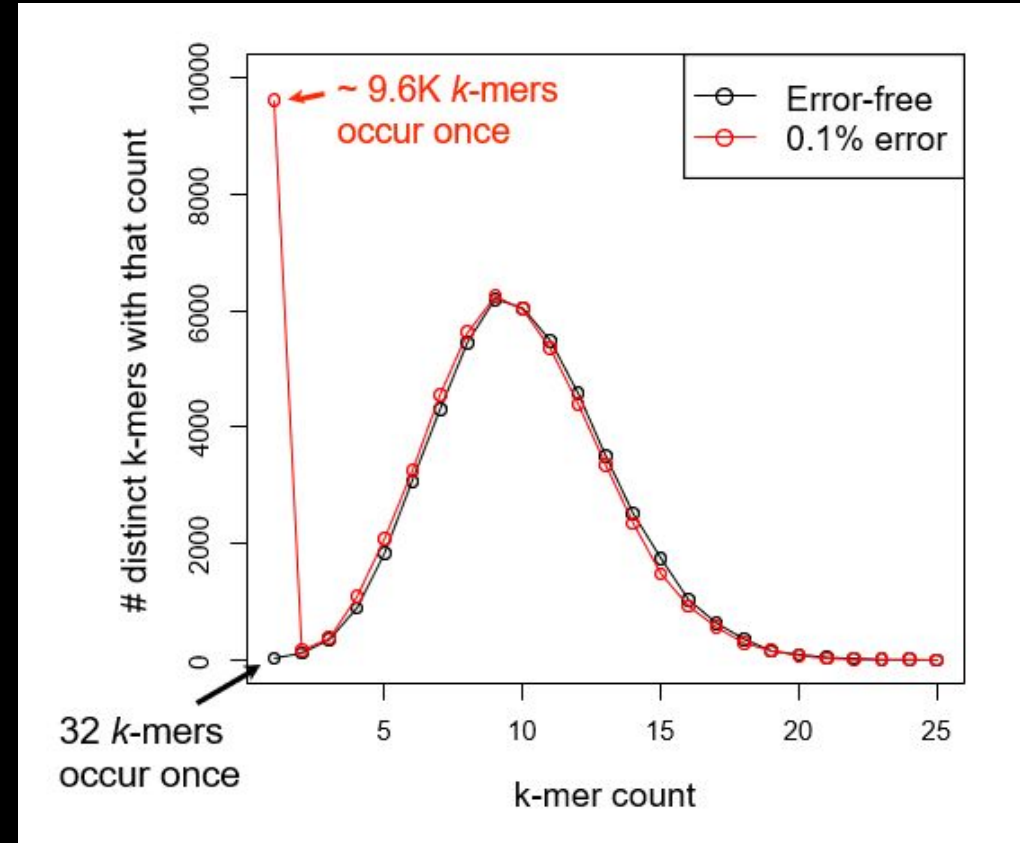


A Big Clue

Larger weights to make the problem a bit more interesting:

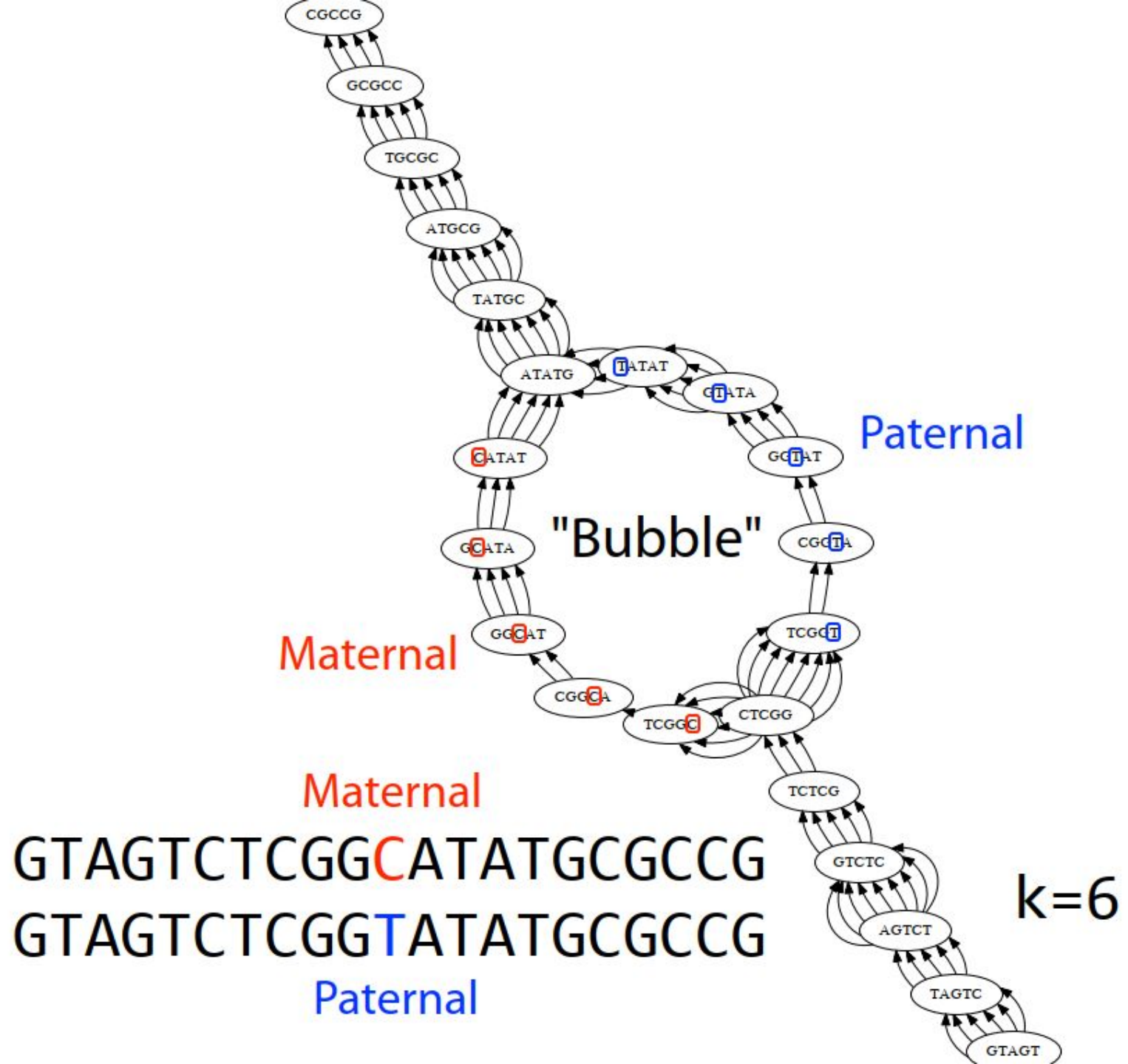
k -mers that arise due to sequencing error tend to be very rare

...and occupy weird spaces in the graph (disconnected nodes as “islands”, alternative entry/exit points as “tips”)



[illegible]

True variations
due for (for
example) ploidy
or differences in
the population
often appear as
“bubbles”

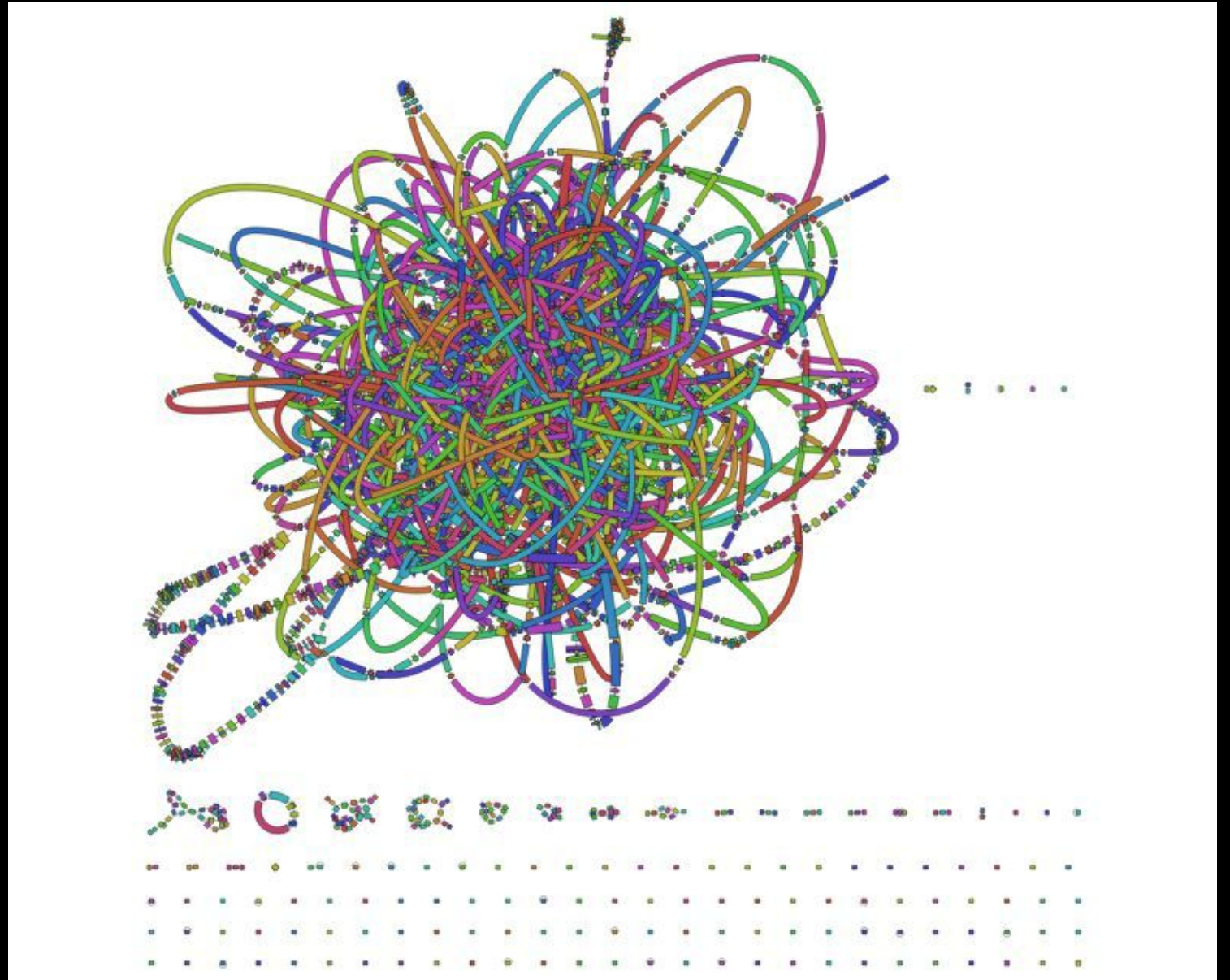


The Million-Nucleotide Question

Which k to choose?

(brace yourself...real genome data incoming)

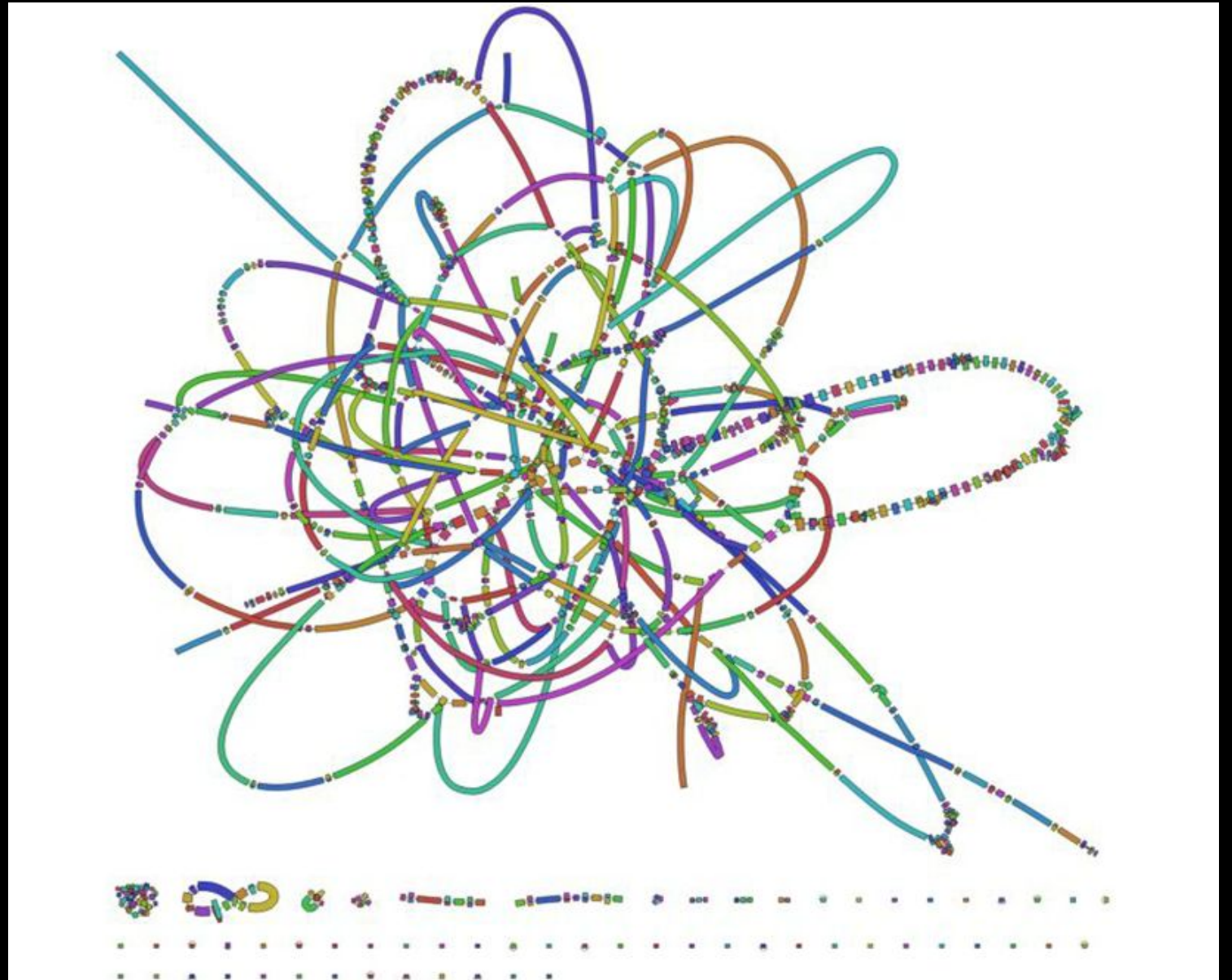
$k = 51$:
4618 nodes, 6070 edges



Salmonella genome assembly of 100bp Illumina reads

<https://github.com/rrwick/Bandage/wiki/Effect-of-kmer-size>

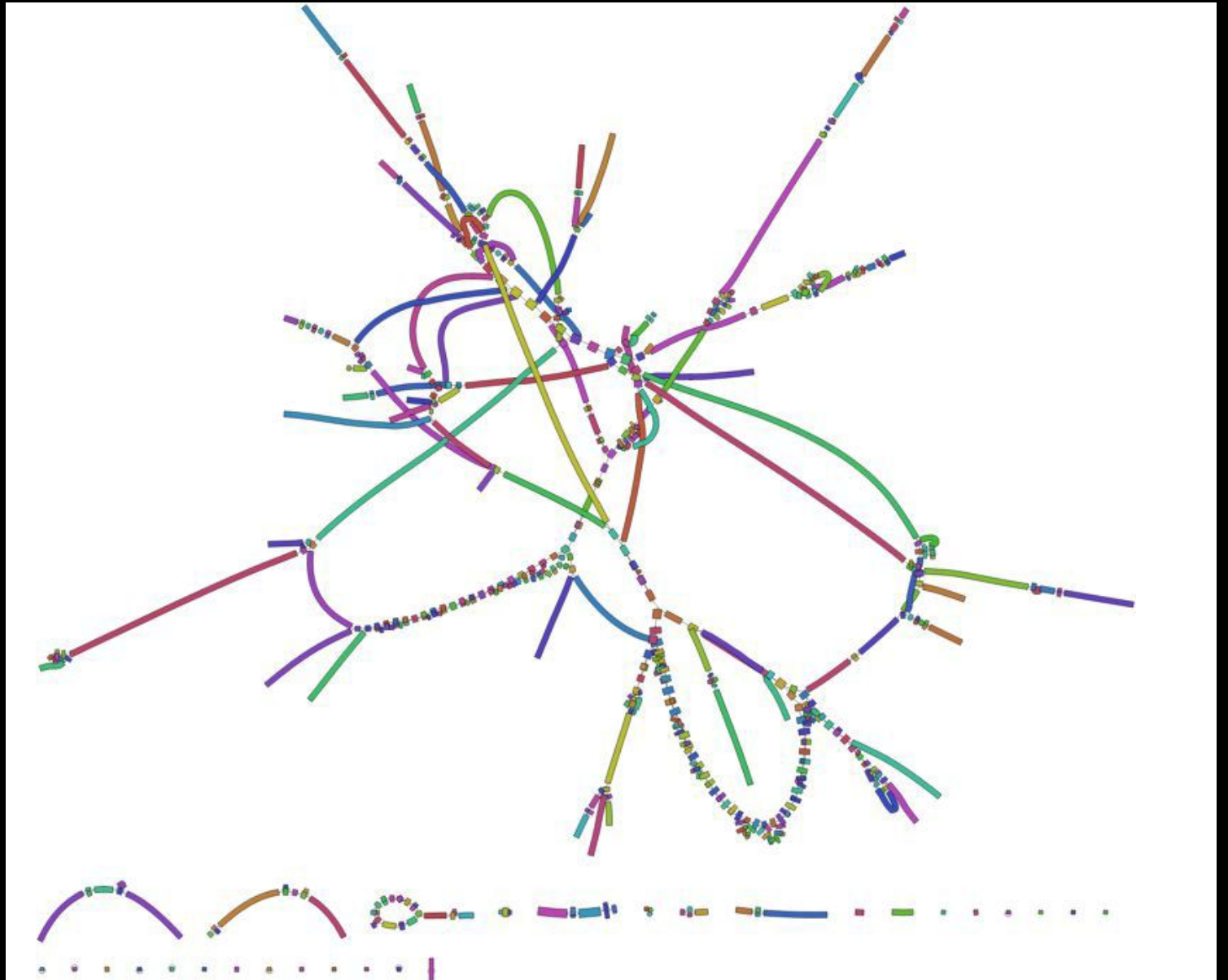
$k = 61$:
1357 nodes, 1768 edges



Salmonella genome assembly of 100bp Illumina reads

<https://github.com/rrwick/Bandage/wiki/Effect-of-kmer-size>

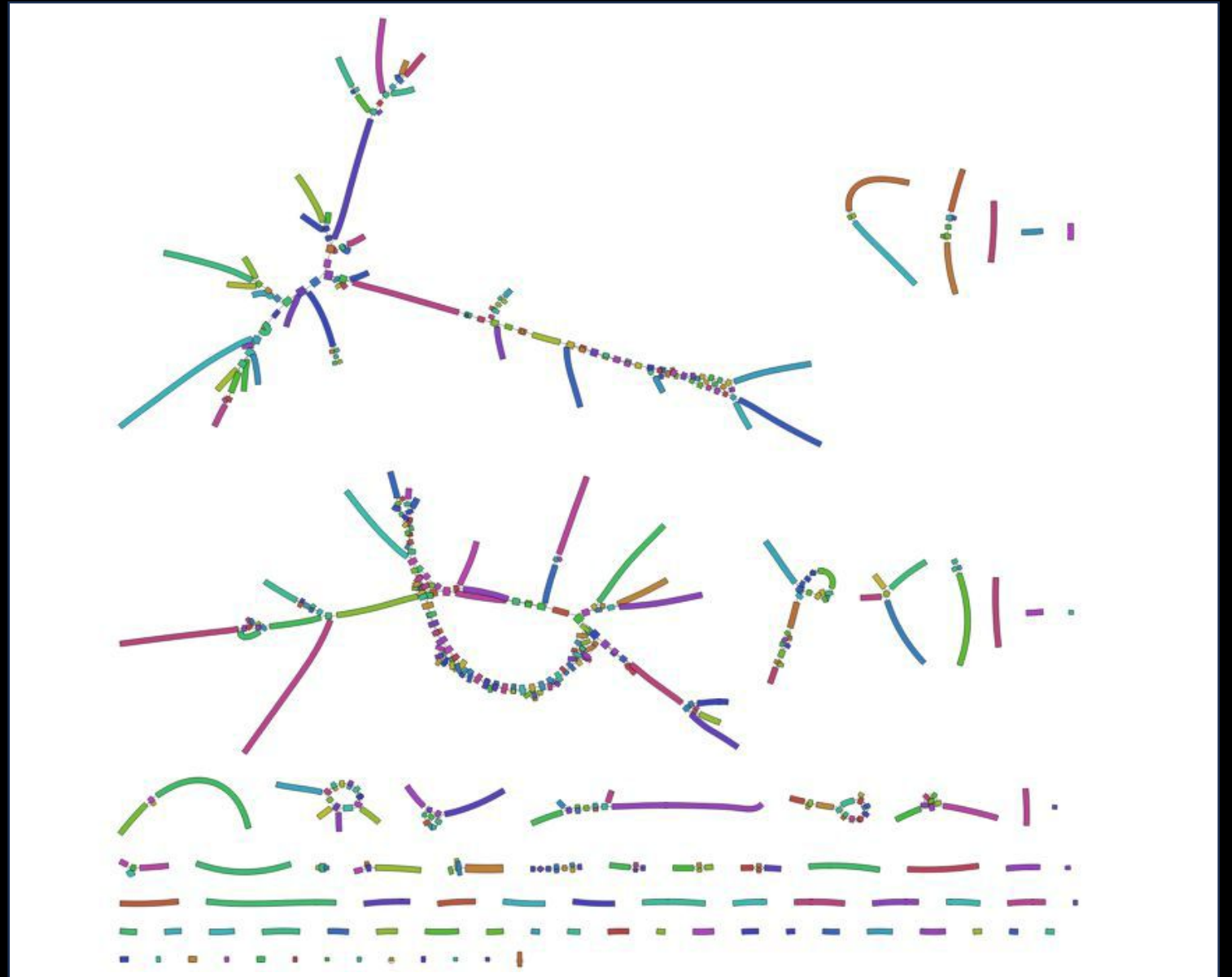
$k = 71$:
611 nodes, 765 edges



Salmonella genome assembly of 100bp Illumina reads

<https://github.com/rswick/Bandage/wiki/Effect-of-kmer-size>

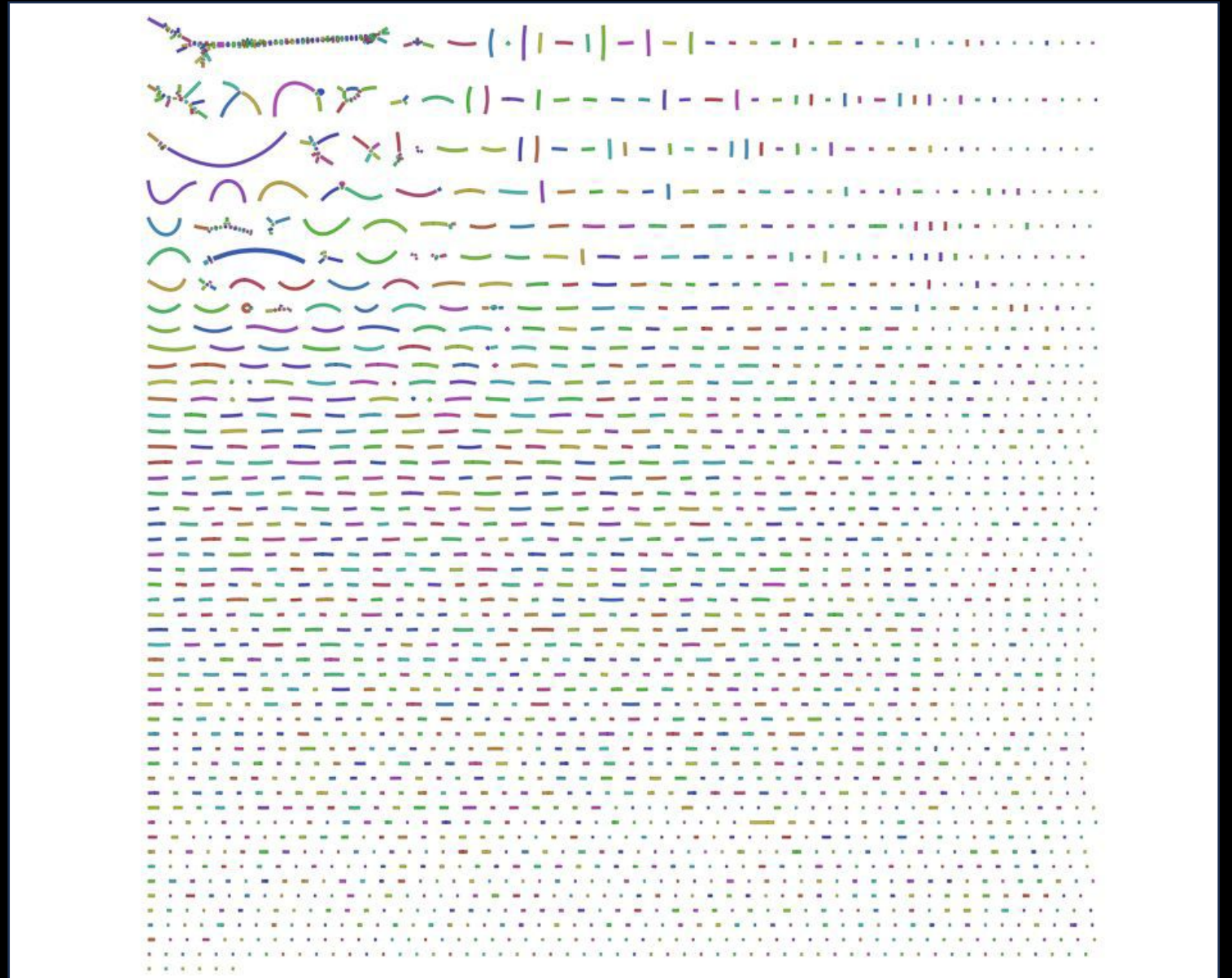
$k = 81$:
490 nodes, 512 edges



Salmonella genome assembly of 100bp Illumina reads

<https://github.com/rswick/Bandage/wiki/Effect-of-kmer-size>

$k = 91$:
2386 nodes, 304 edges



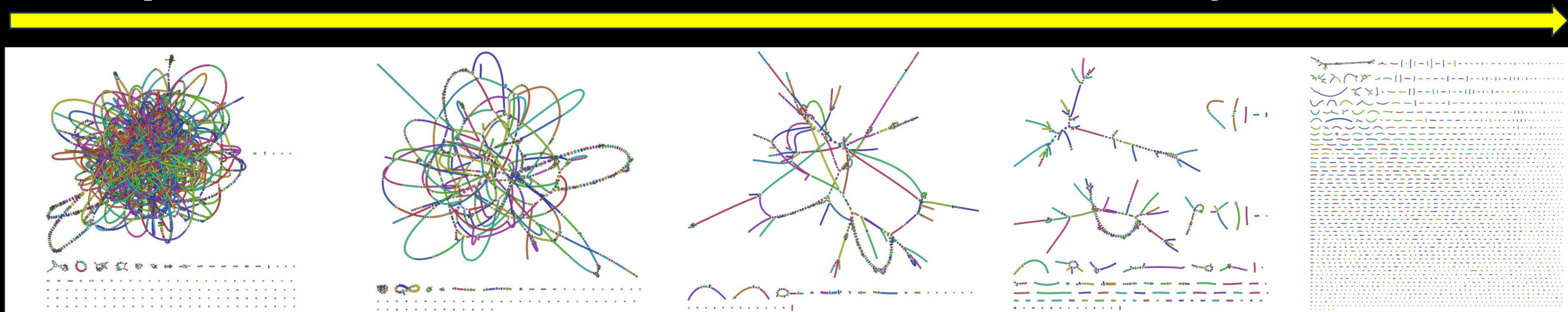
Salmonella genome assembly of 100bp Illumina reads

<https://github.com/rrwick/Bandage/wiki/Effect-of-kmer-size>

What Is Going On Here?

k small: spurious connections

k large: no connections at all!



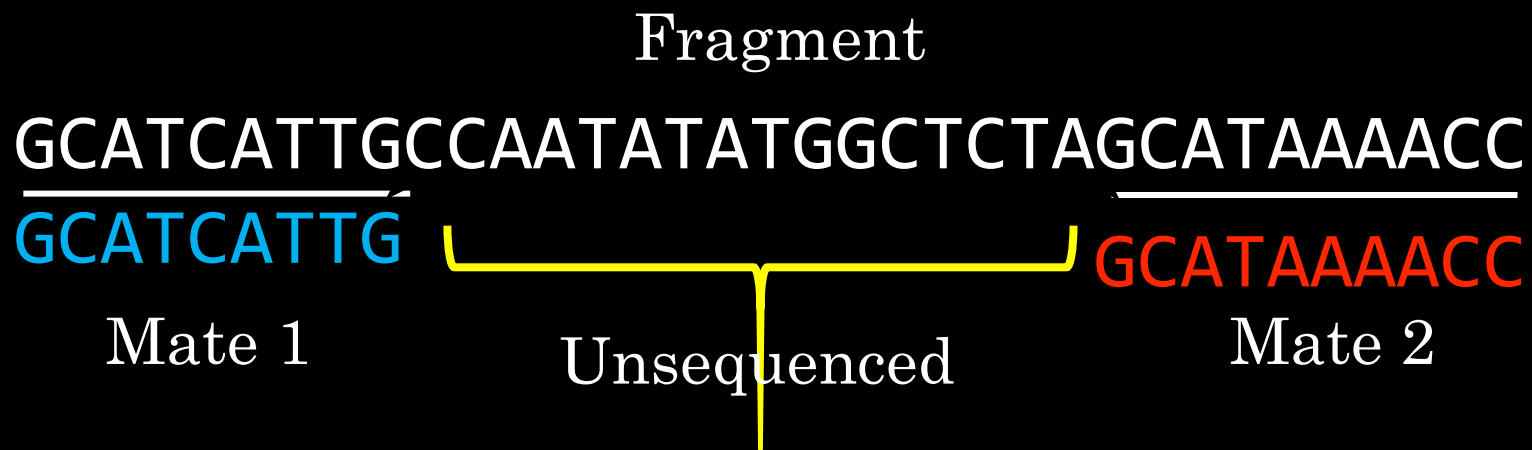
SPAdes assembler: Try many values of k and reconcile the results

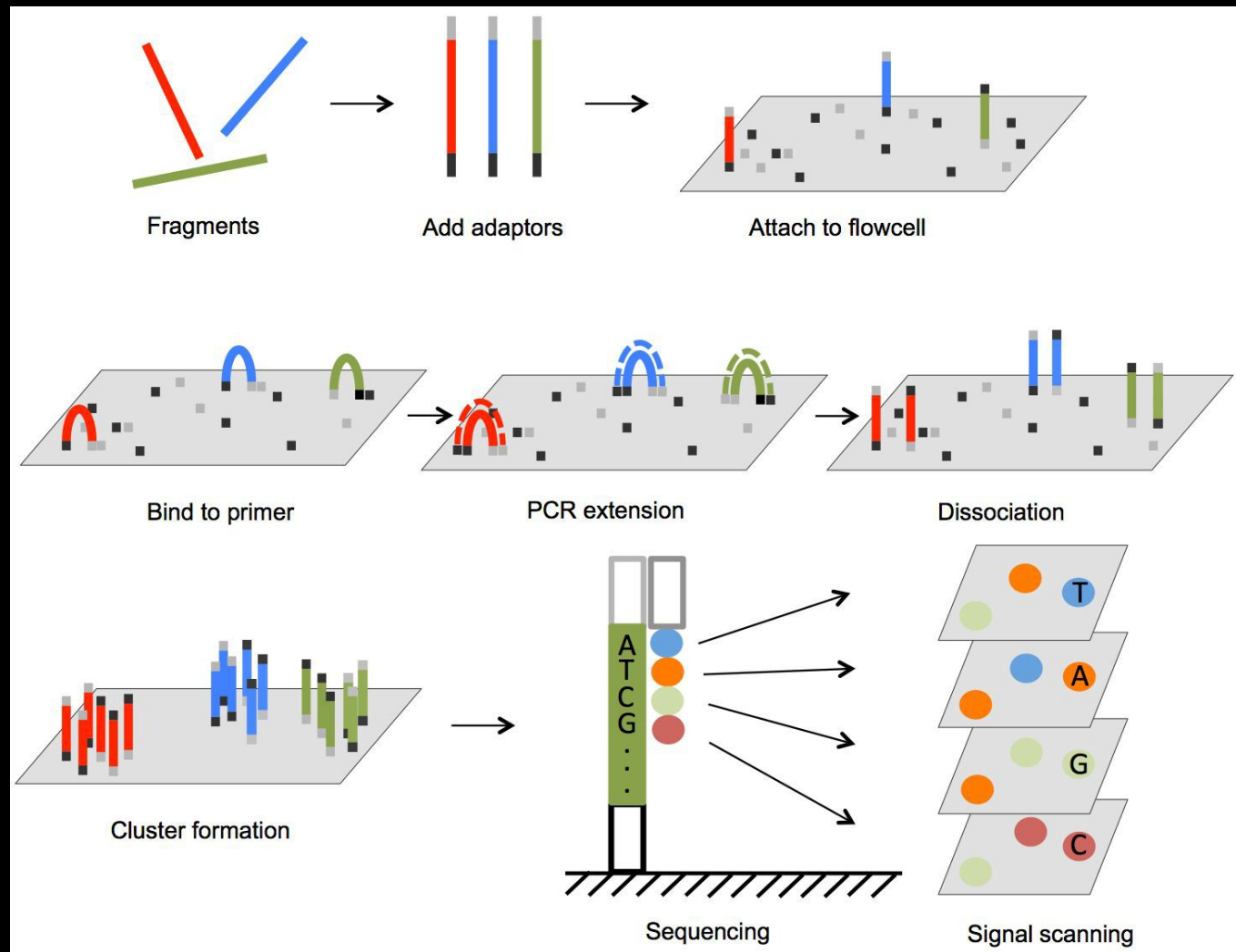
Scaffolding

- Assembly gives us **contigs**, stretches of unambiguous assembled sequence
- Whatever method we use, we will probably not get a complete genome / chromosome / etc 😞
- BUT, we can often draw on other contextual information to orient contigs with respect to one another on **scaffolds**

Paired-End Sequencing

- So far, we have considered sequencing and DNA fragment or one end of it
- But in many cases (especially Illumina sequencing by synthesis), we can sequence both ends of a given DNA fragment, giving us **paired-end reads**





GCATCATTGCCAATATATGGCTCTAGCATAAAACC

GCATCATTG

Mate 1

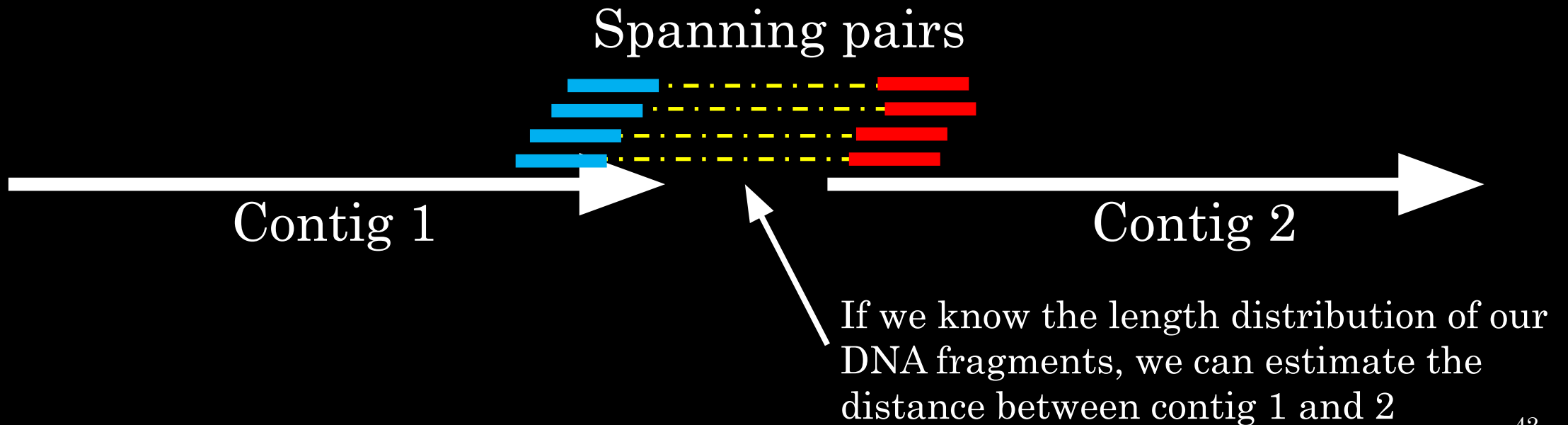
Unsequenced

GCATAAAACC

Mate 2

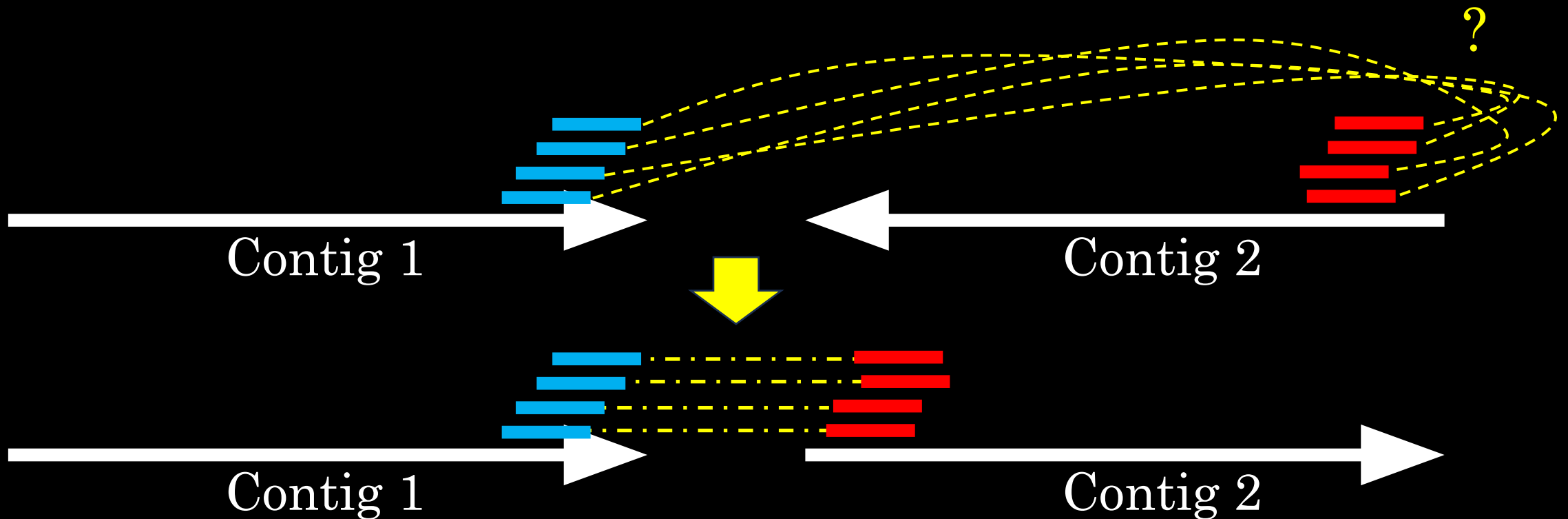
Scaffolding From Paired Ends

- Say we have a collection of pairs and we assemble them as usual
- Assembly yields two contigs
- These contigs may be **spanned** by mate pairs



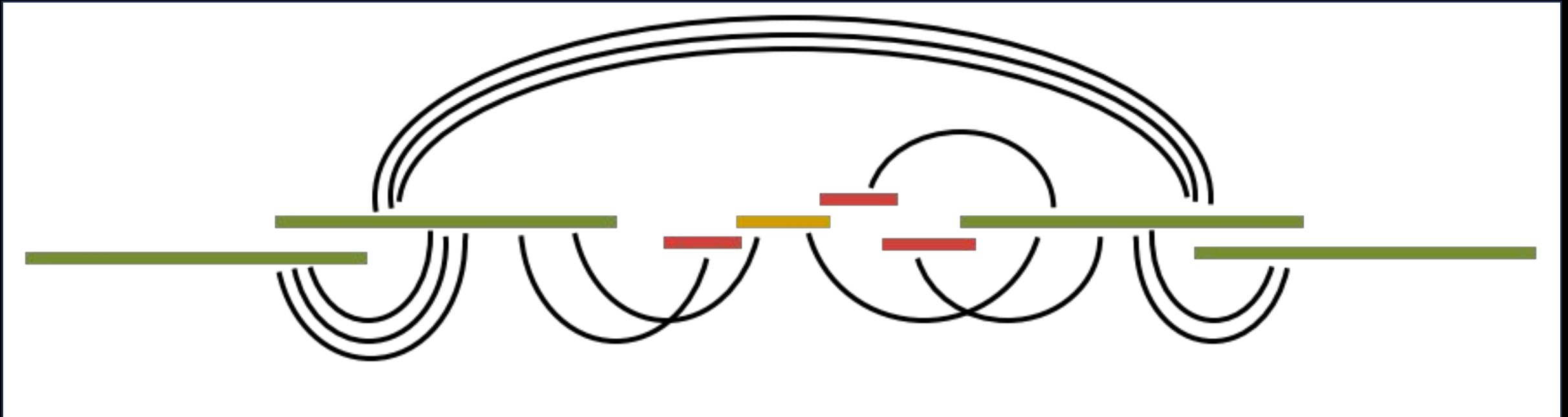
Orienting Contigs

- If one contig is “backwards” (i.e., reverse complemented), spanning pairs can give us the correct orientation



Scaffolds

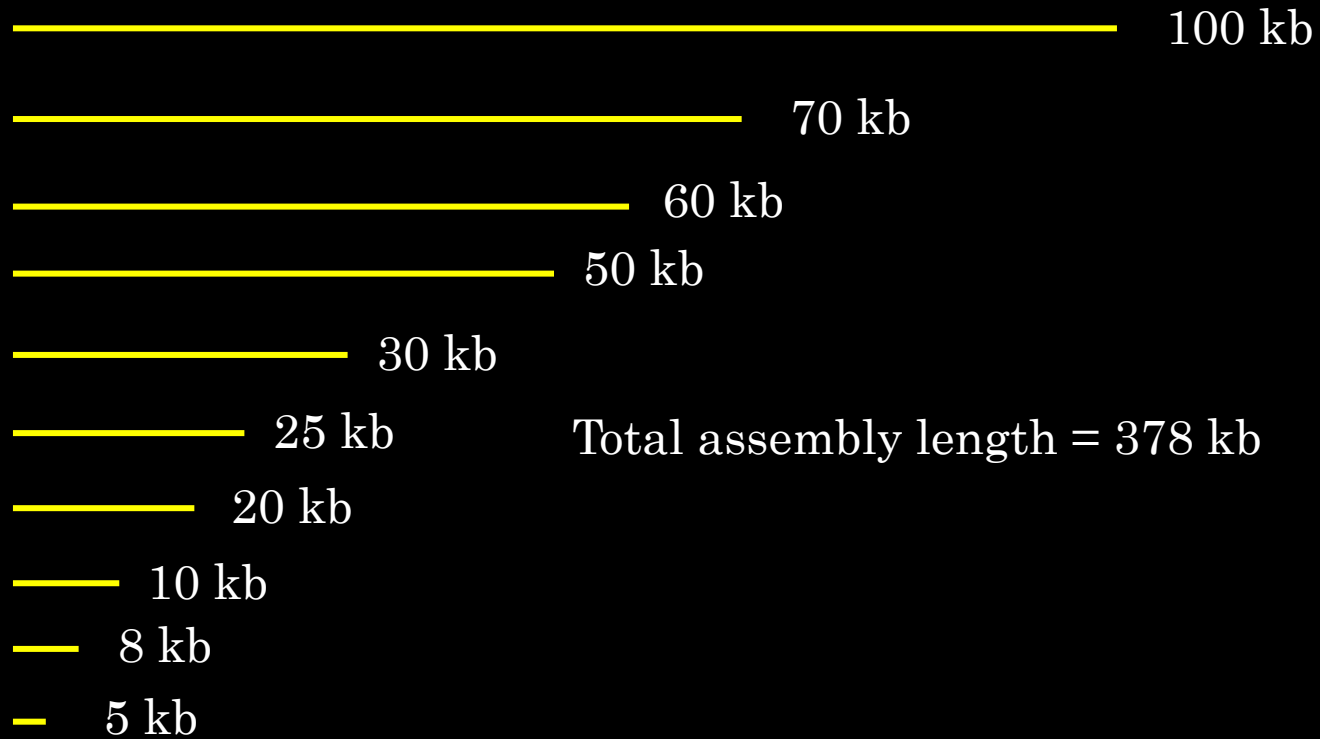
Scaffolding output: collection of scaffolds, where a scaffold is a collection of contigs related to each other with high confidence using pairs



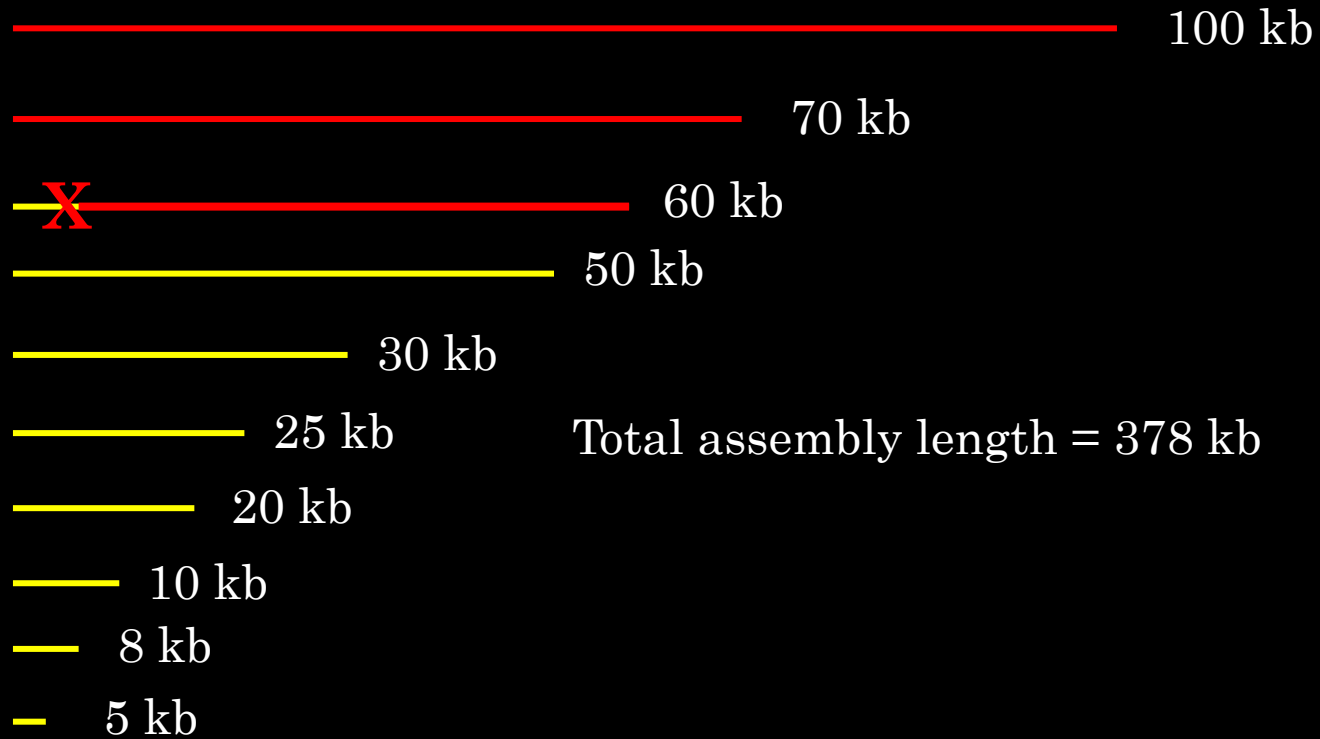
Soooo...how good is my assembly?

- Various assembly statistics:
 - Total genome length
 - Number of scaffolds / contigs
 - Contig size
 - Presence of expected “core” genes
 - Presence of contamination
- Can be completely naïve, or use reference knowledge about a specific taxonomic group (“*E. coli* genomes are about this big”)

Example Statistics



Example Statistics



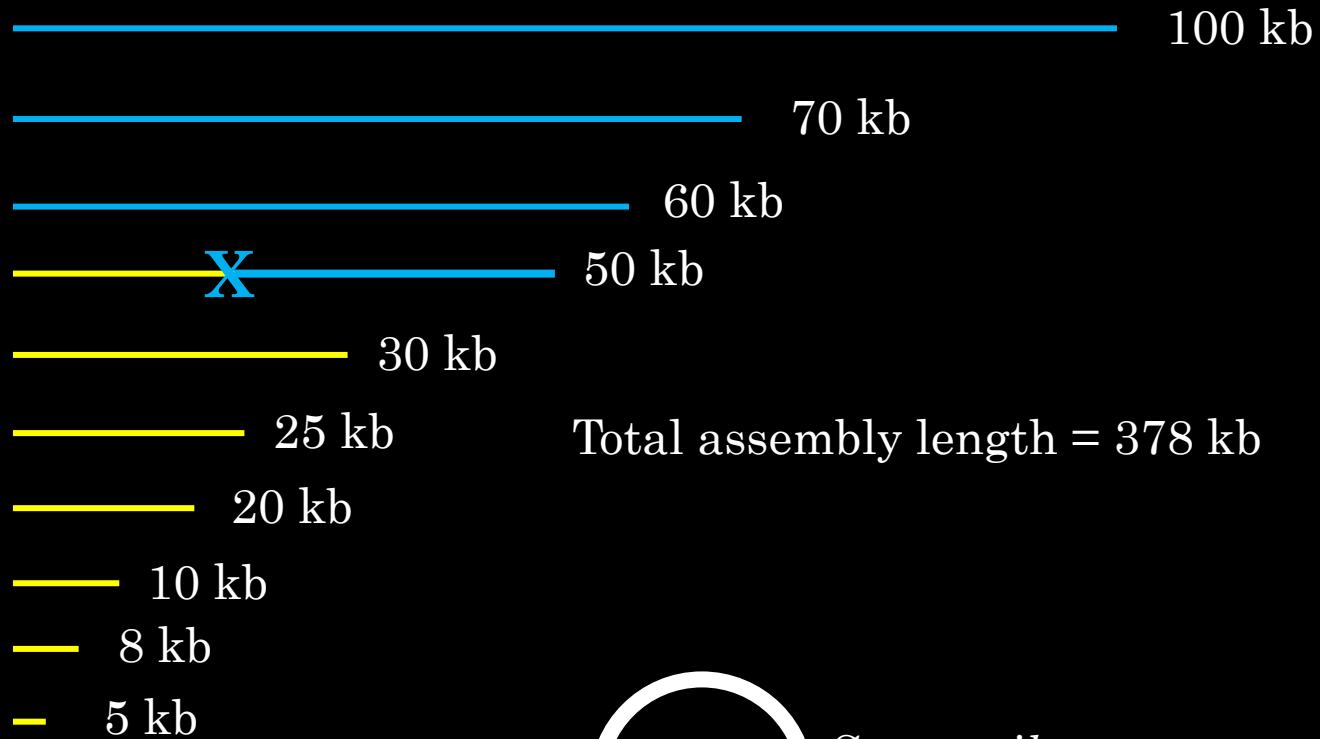
N50 – The length of the contig that contains the “median nucleotide”

$X = 189,000$

Contig containing X is 60 kb

N50 = 60

Example Statistics



NG50 – The length of the contig that contains the “**median nucleotide**” relative to *some expected genome size*

X = 250,000

Contig containing **X** is 50 kb

NG50 = 50

Download ▾		Select columns		39,372 Genomes		Rows per page		20 ▾		1-20 of 39,372		◀	
☐	Scientific name	↑ Size (Mb)	Level	Contig N50 (kb)	CheckM completeness ...	CheckM contamination ...							
☐	Pseudomonas aeruginosa	0.04193	Chromosome	41.93	0	0							
☐	Pseudomonas aeruginosa	0.06010	Chromosome	60.09	0	0							
☐	Pseudomonas aeruginosa	0.06571	Complete	65.71	0	0							
☐	Pseudomonas aeruginosa	0.07278	Complete	72.78	0	0							
☐	Pseudomonas aeruginosa	0.1017	Contig	38.84	0	0							
☐	Pseudomonas aeruginosa	12.23	Contig	9.80	0	0							
☐	Pseudomonas aeruginosa	11.97	Contig	8.44	0	0							
☐	Pseudomonas aeruginosa	11.85	Contig	28.04	0	0							
☐	Pseudomonas aeruginosa	10.66	Scaffold	274.04	0	0							
☐	Pseudomonas aeruginosa	9.805	Scaffold	193.91	0	0							
☐	Pseudomonas aeruginosa	7.601	Complete	7,137.03	99.39	2.27							
☐	Pseudomonas aeruginosa	7.584	Complete	7,408.98	99.11	1.48							
☐	Pseudomonas aeruginosa	7.564	Complete	7,564.38	97.65	3.53							
☐	Pseudomonas aeruginosa	7.557	Complete	7,094.52	99.05	3.15							

NCBI Genomes Database:
~39,000 “genomes” of
Pseudomonas aeruginosa

Unrealistically small!

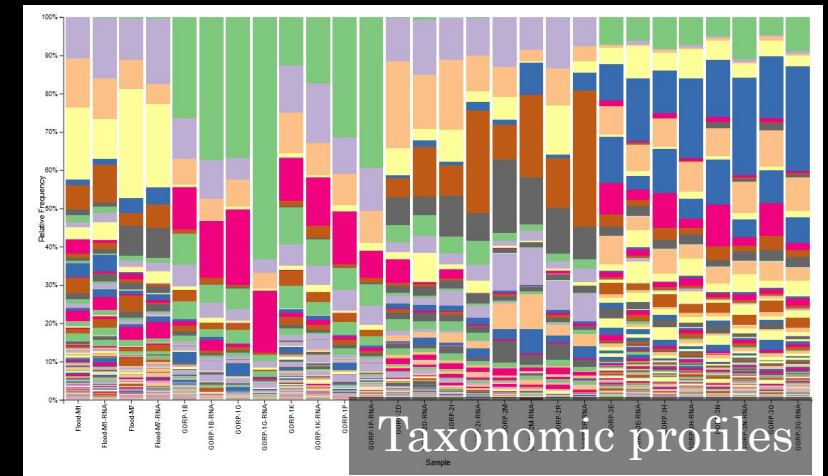
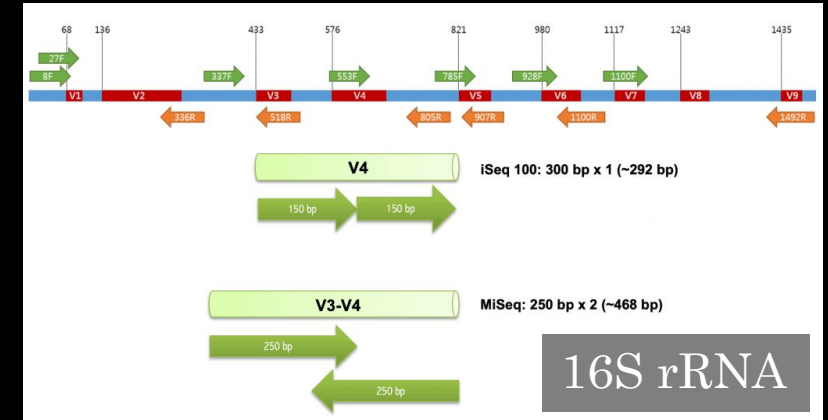
Unrealistically large!

Size range of 904 complete,
annotated genomes (i.e., not
just a bunch of contigs or
scaffolds): 5.9 – 7.8
megabases

Profiling Many Organisms At Once

Method 1: Marker-Gene Profiling

- Choose a gene that is common to all organisms of interest
 - Popular choices are “informational” genes involved in replication, transcription, and translation
- Develop PCR primers that target part or all of that gene
- Sequence the PCR products, map to a database to determine “who is there”



Method 2: Metagenomics

- Shotgun sequencing of environmental samples
- Instead of reads from one organism, we can sequence from several hundred!
- **Advantages:** Can get a more-detailed view of biodiversity, no need for culturing in the lab. Also gives us a profile of functional genes: “Who is doing what”
- **Disadvantages?**



REA

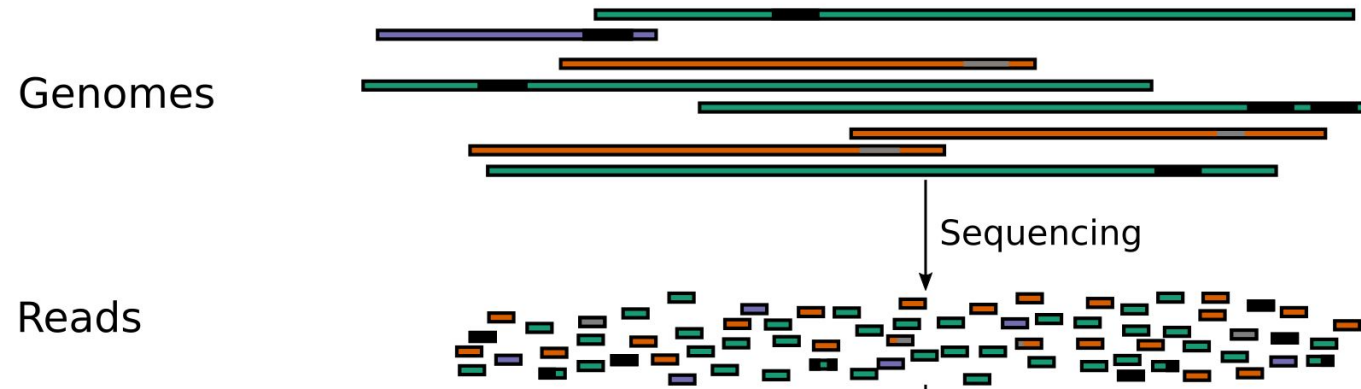
to

Metagenomics

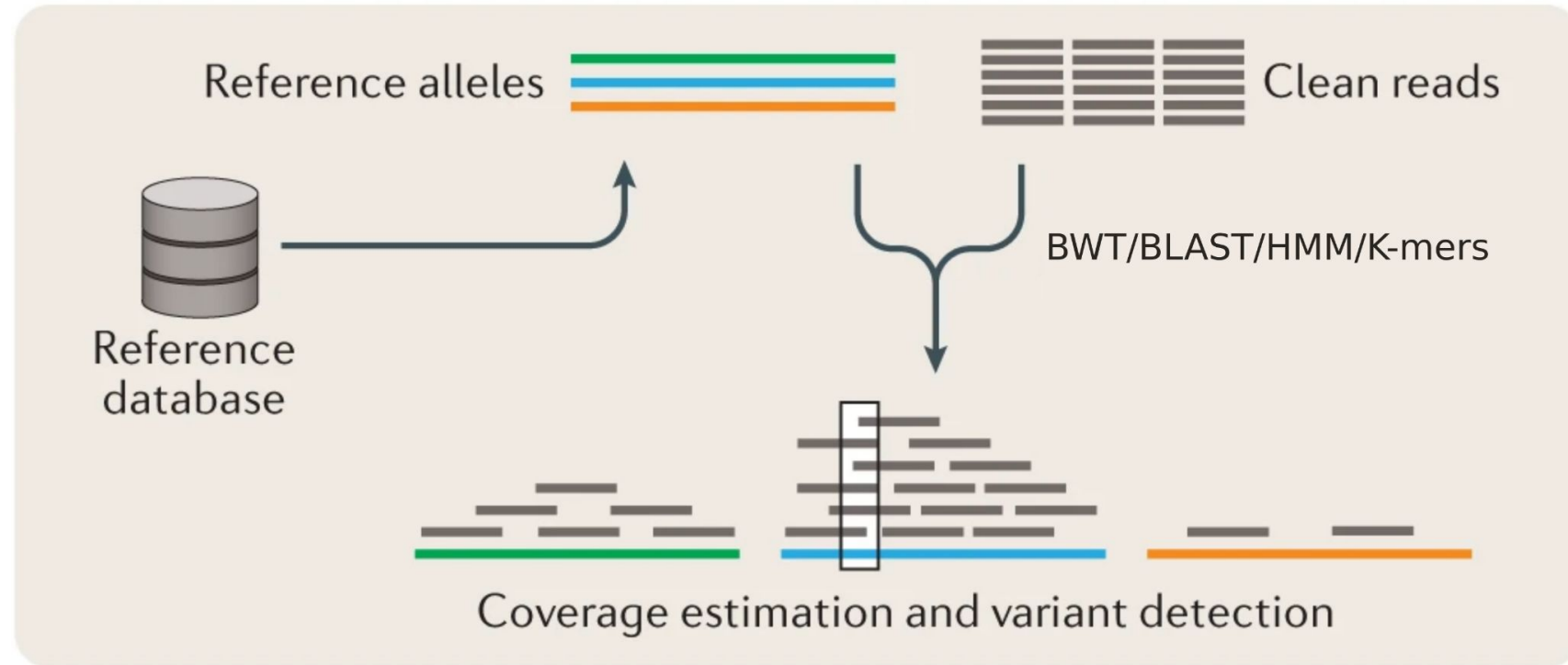
Genomes



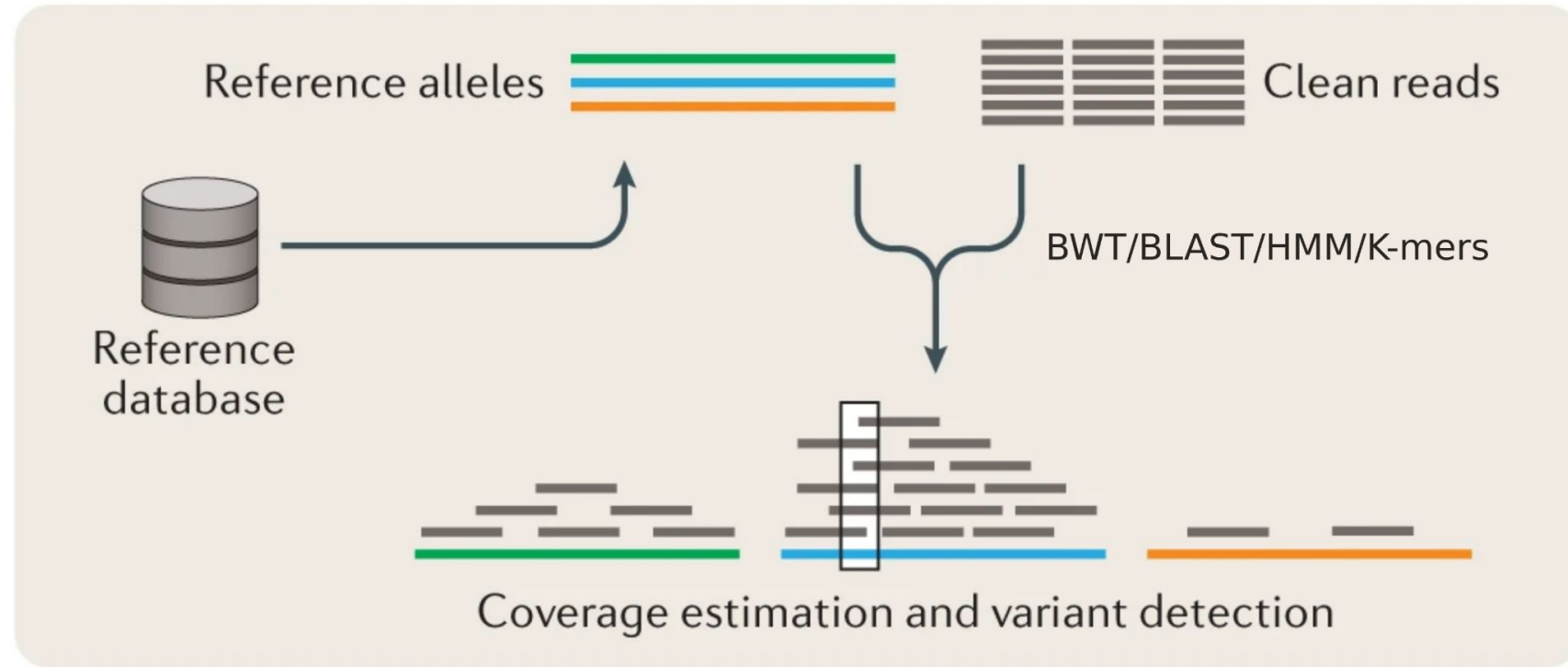
Metagenomics



Read-based analyses



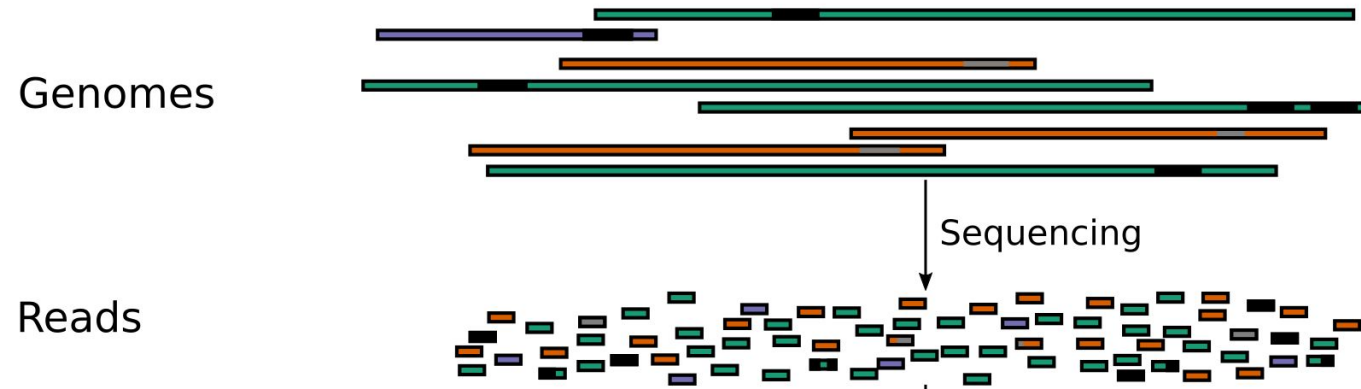
Read-based analyses



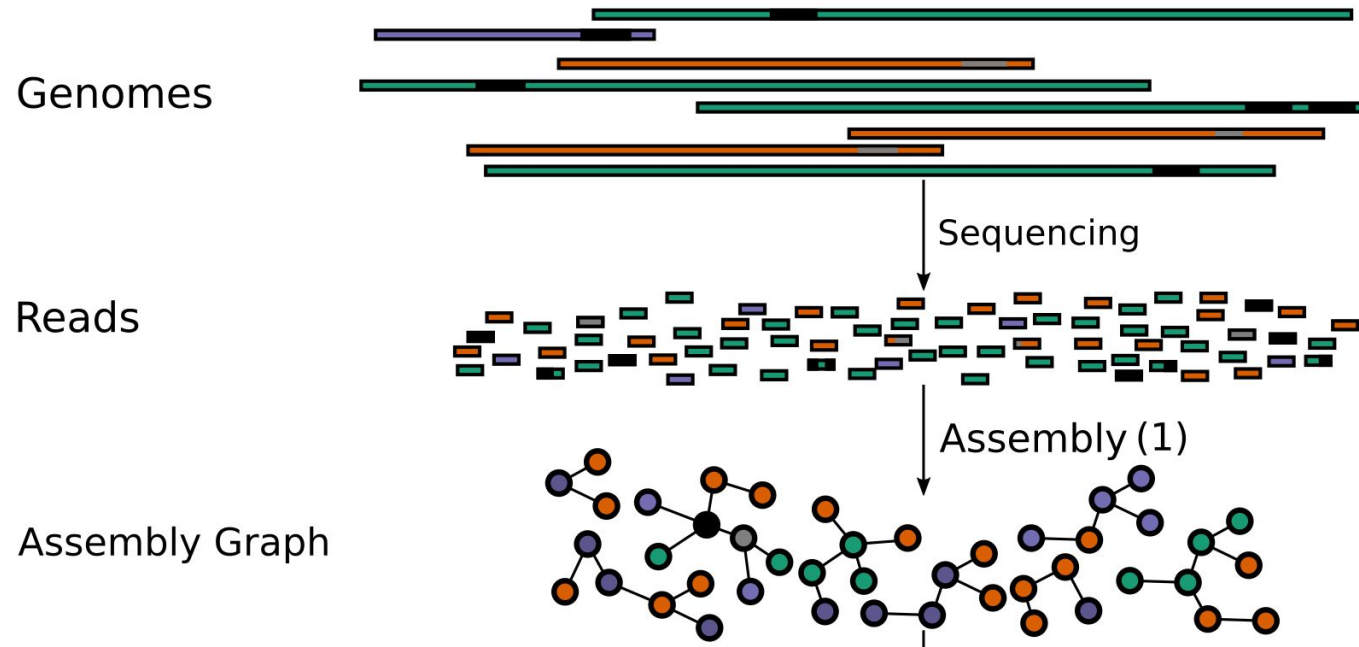
10.1038/s41576-019-0108-4

But - lose wider context, can't resolve alleles, can't find new things!

Metagenomics

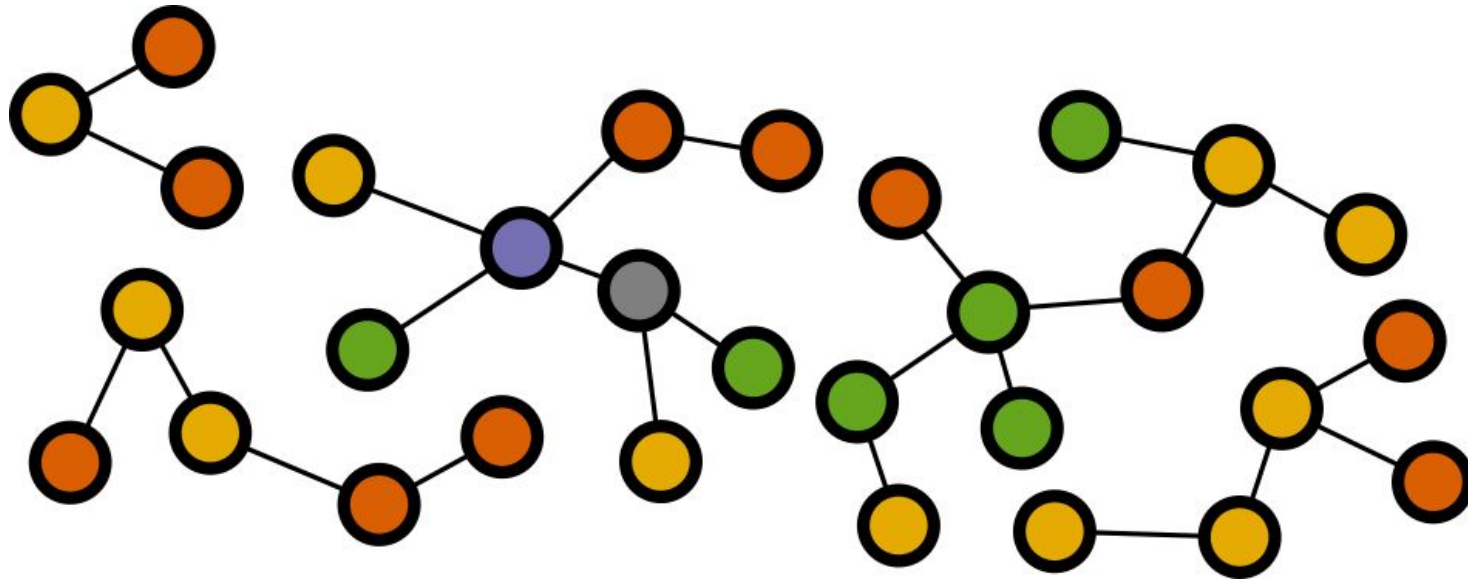


Metagenomics

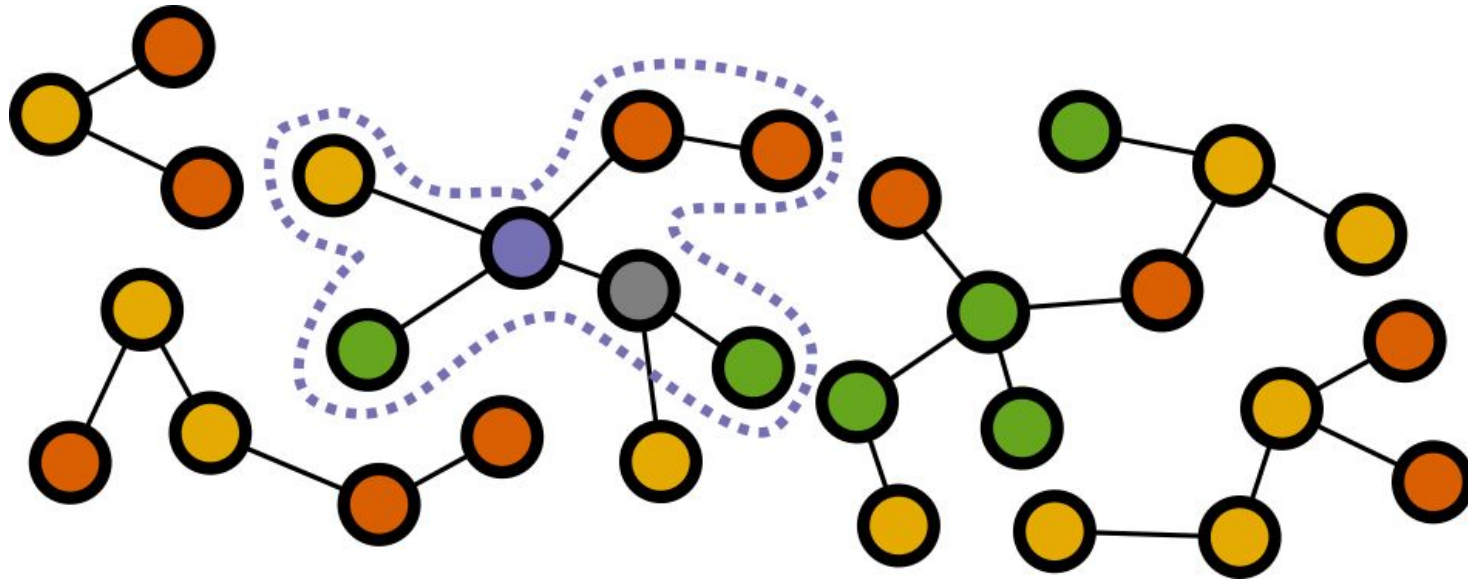


Wildly varied coverage!
How do we resolve repeats,
closely related etc?

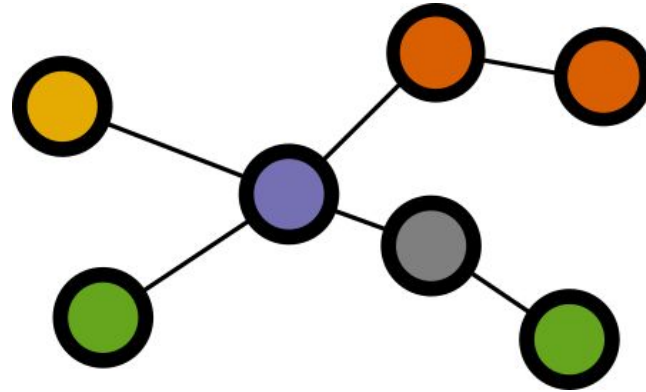
Metagenomic Assembly



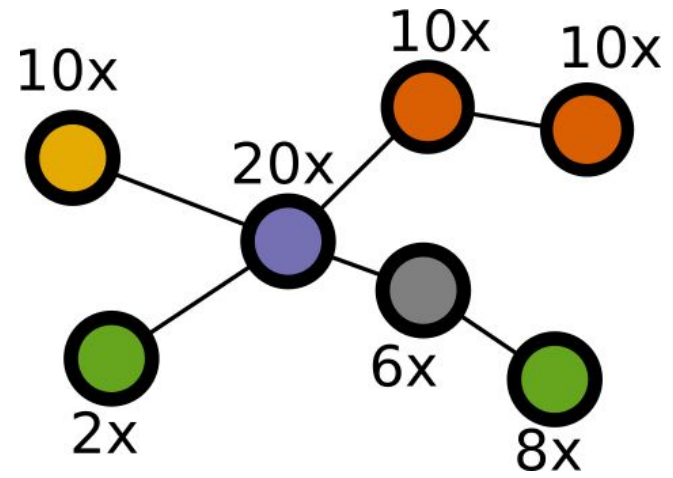
Metagenomic Assembly



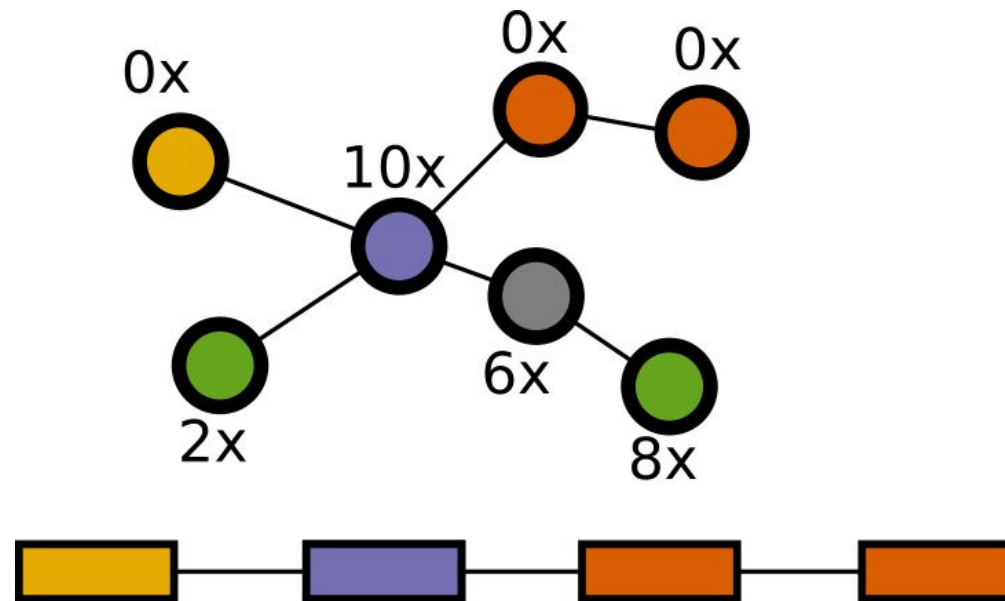
Metagenomic Assembly



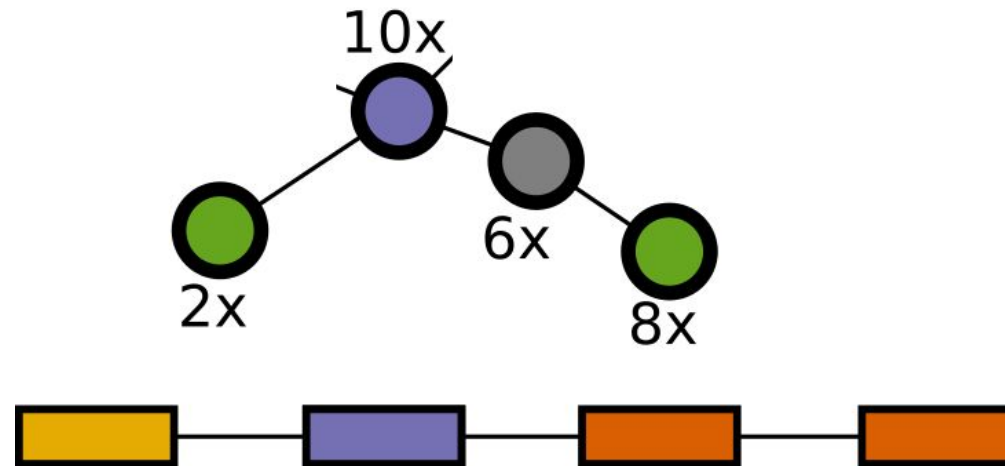
Metagenomic Assembly



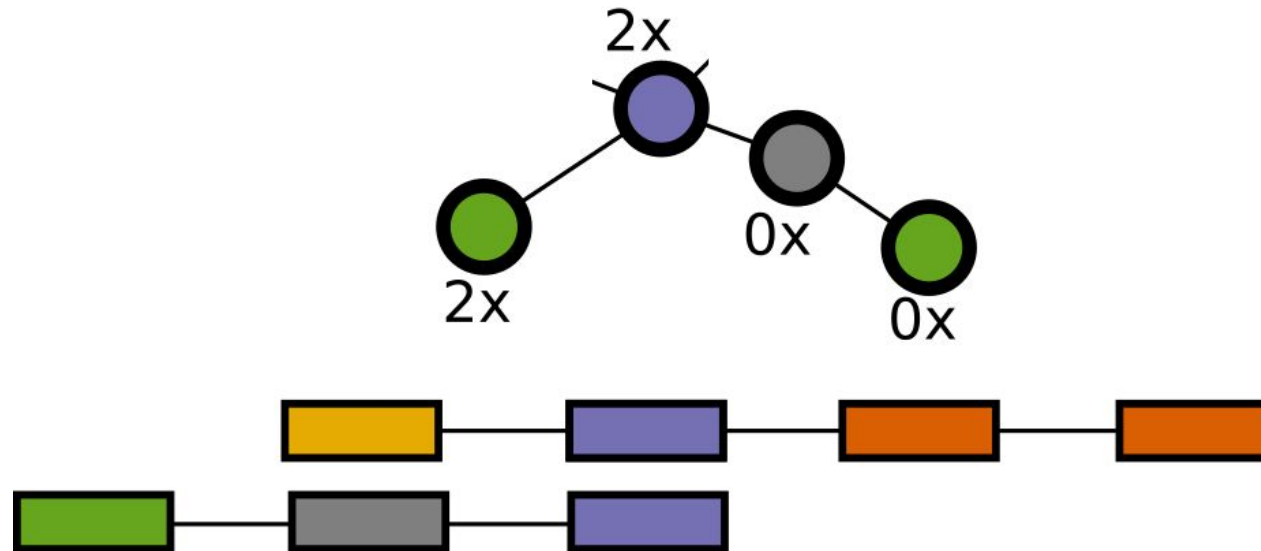
Metagenomic Assembly



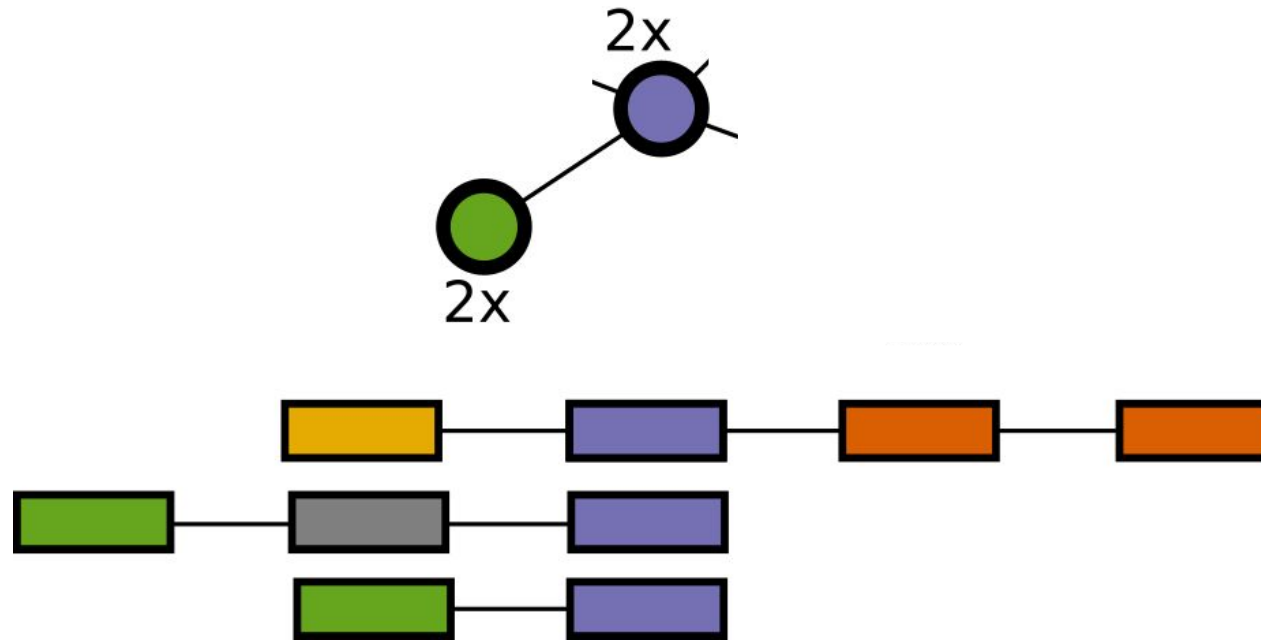
Metagenomic Assembly



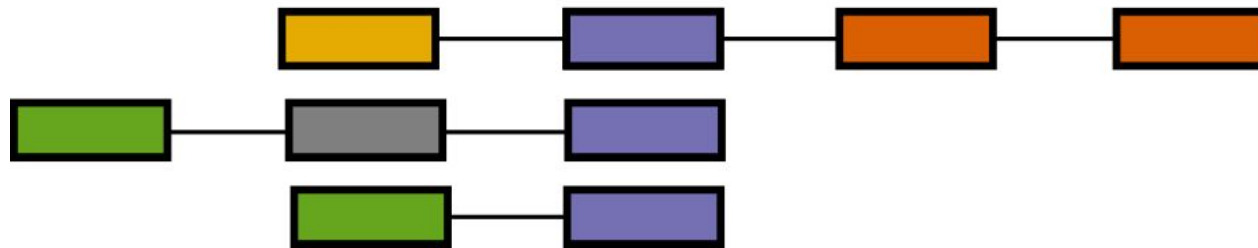
Metagenomic Assembly



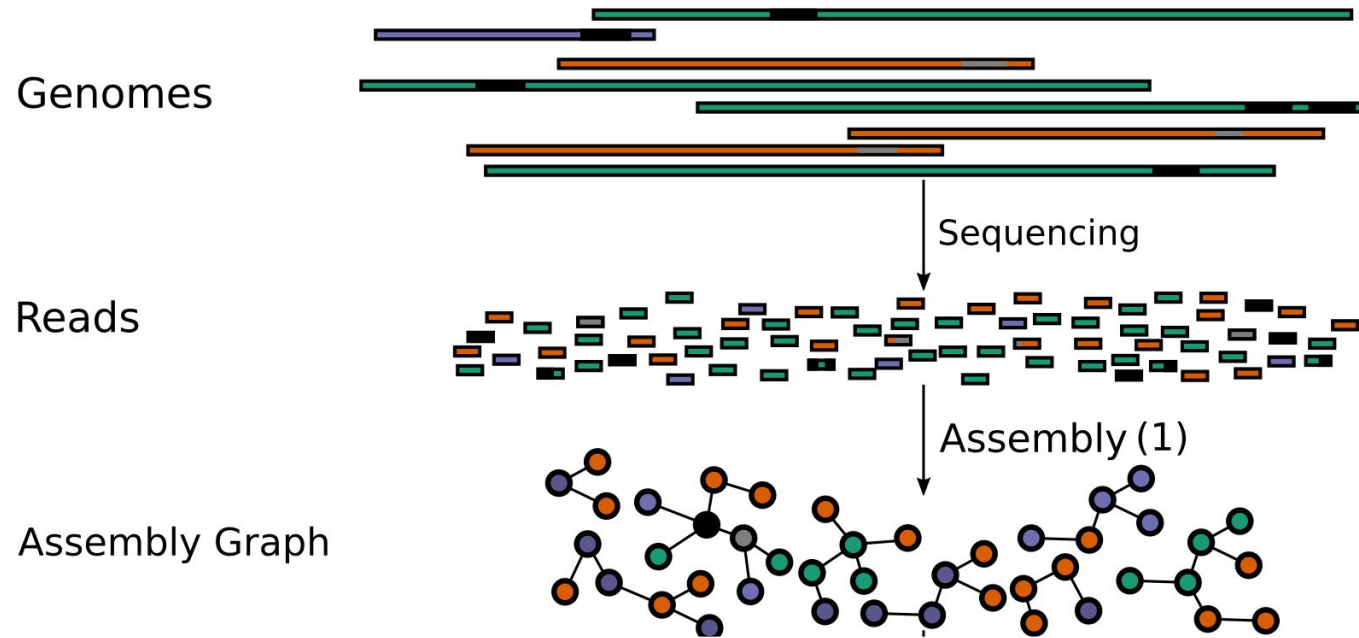
Metagenomic Assembly



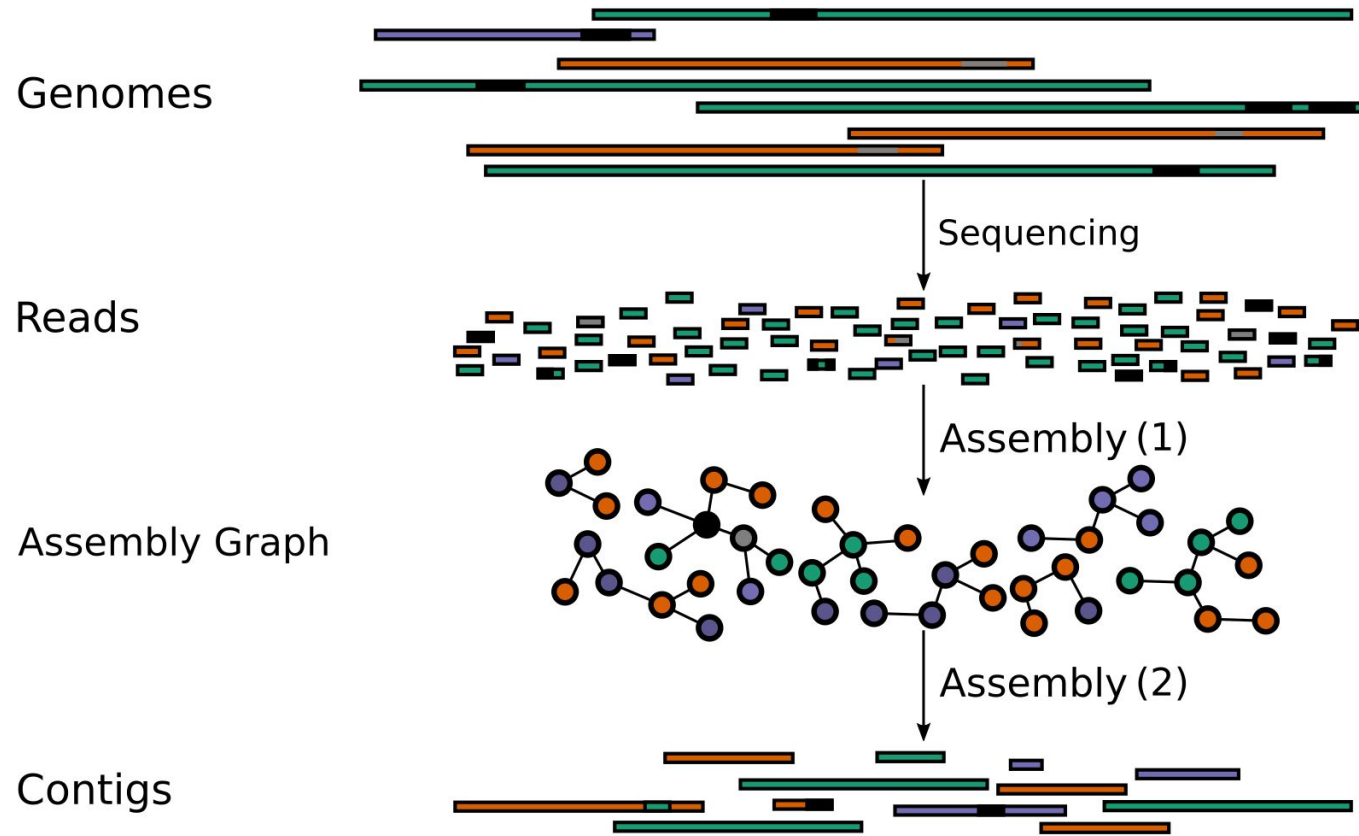
Metagenomic Assembly



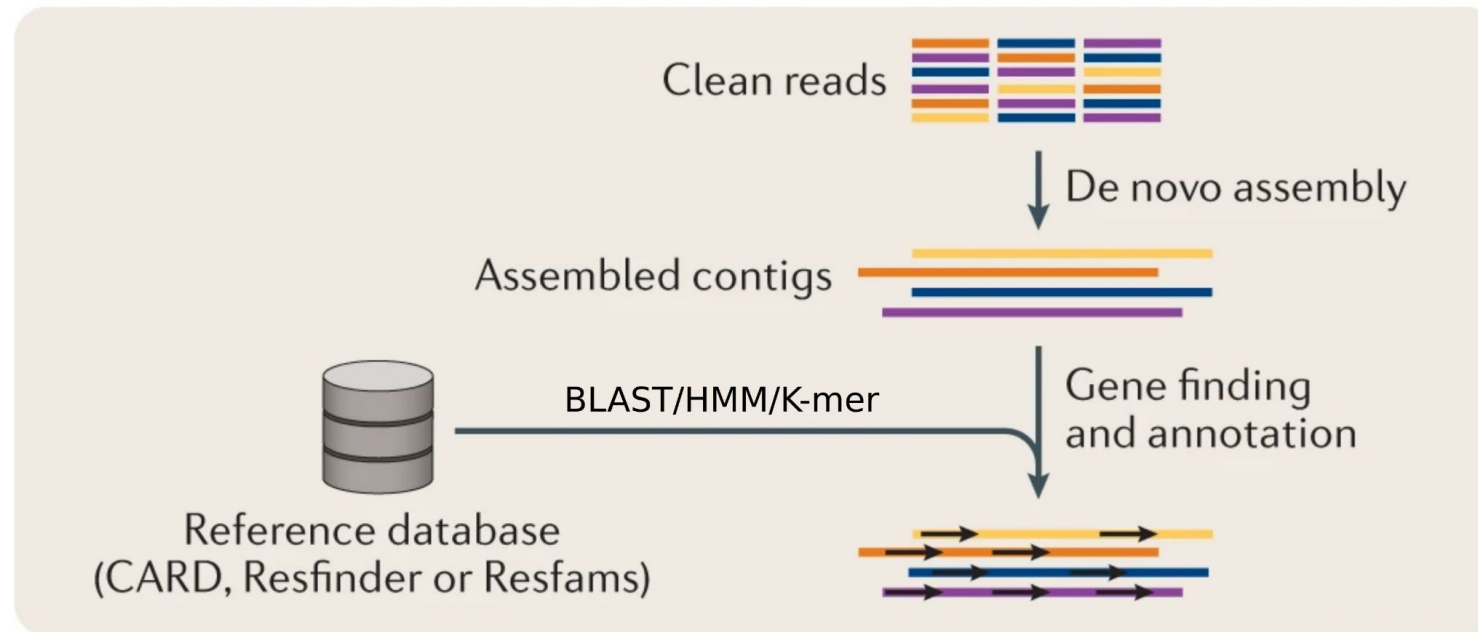
Metagenomics



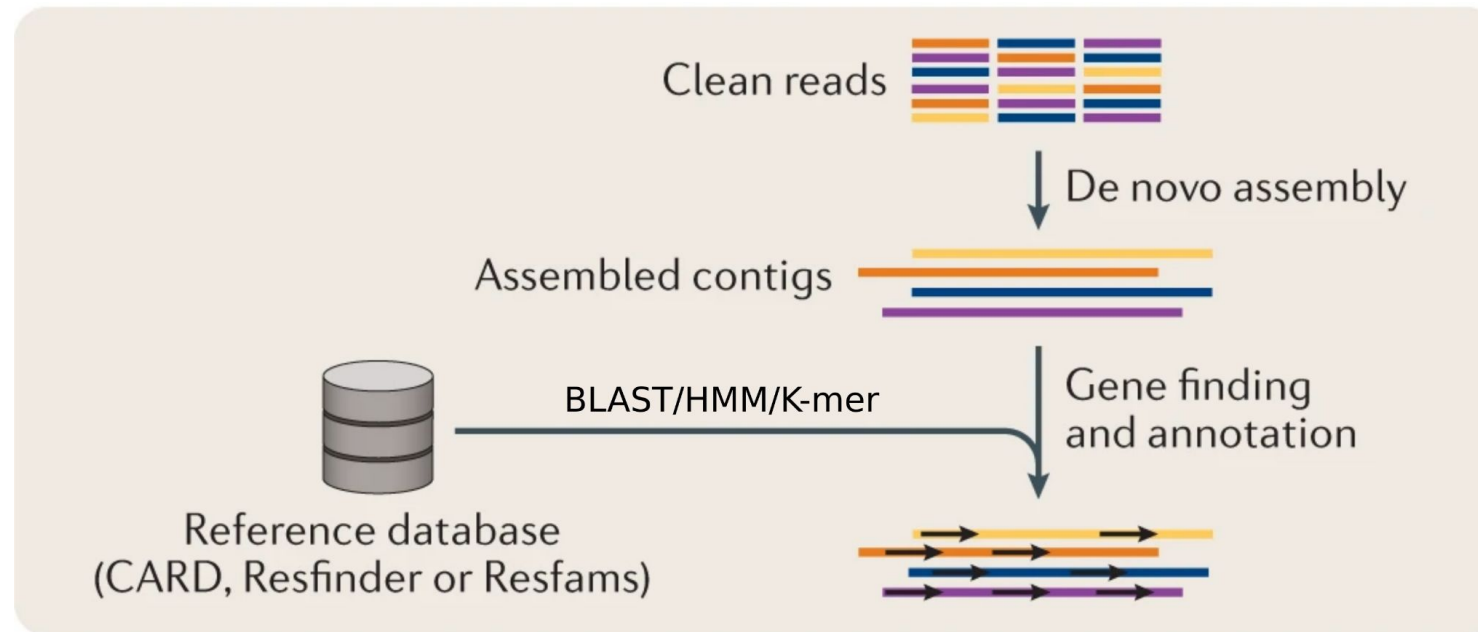
Metagenomics



Contig-based Analyses

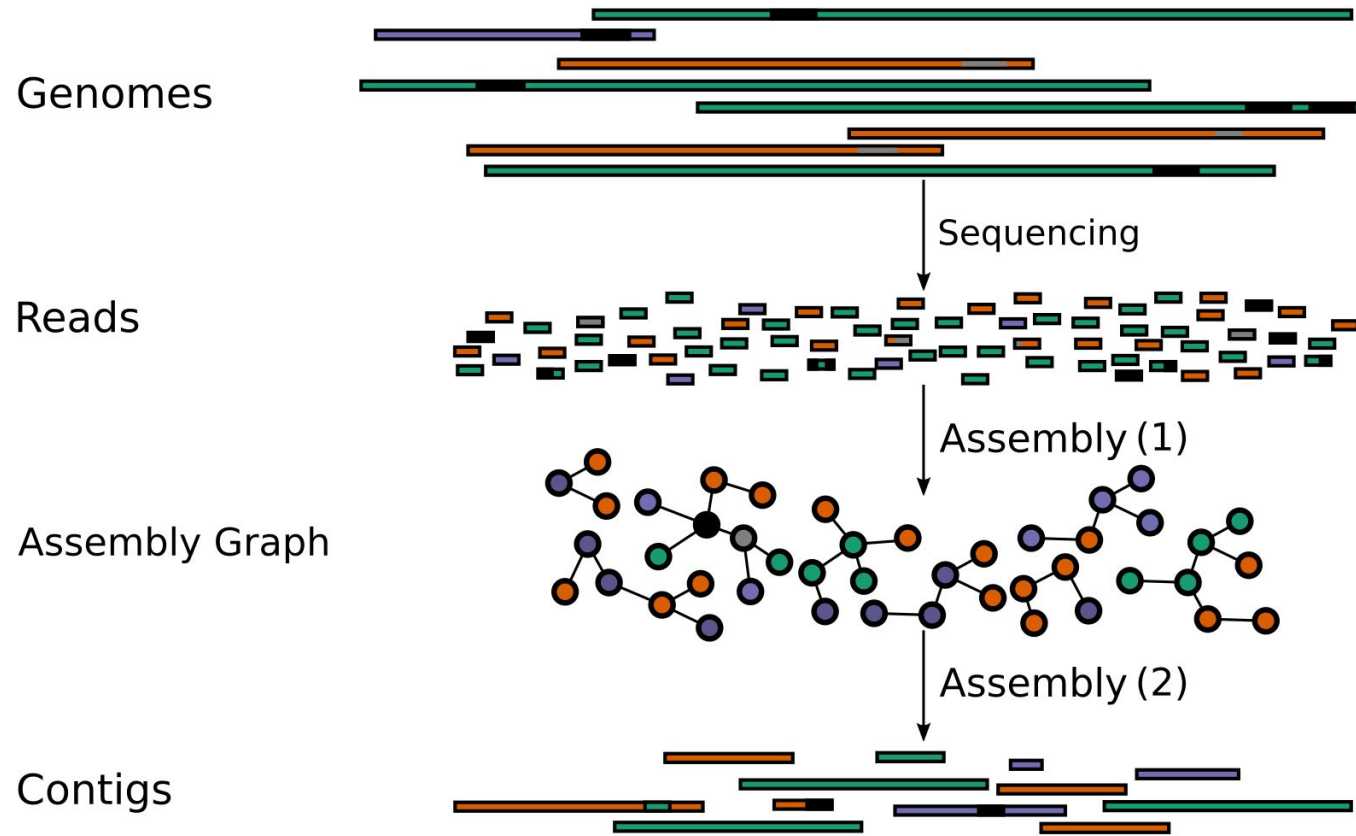


Contig-based Analyses

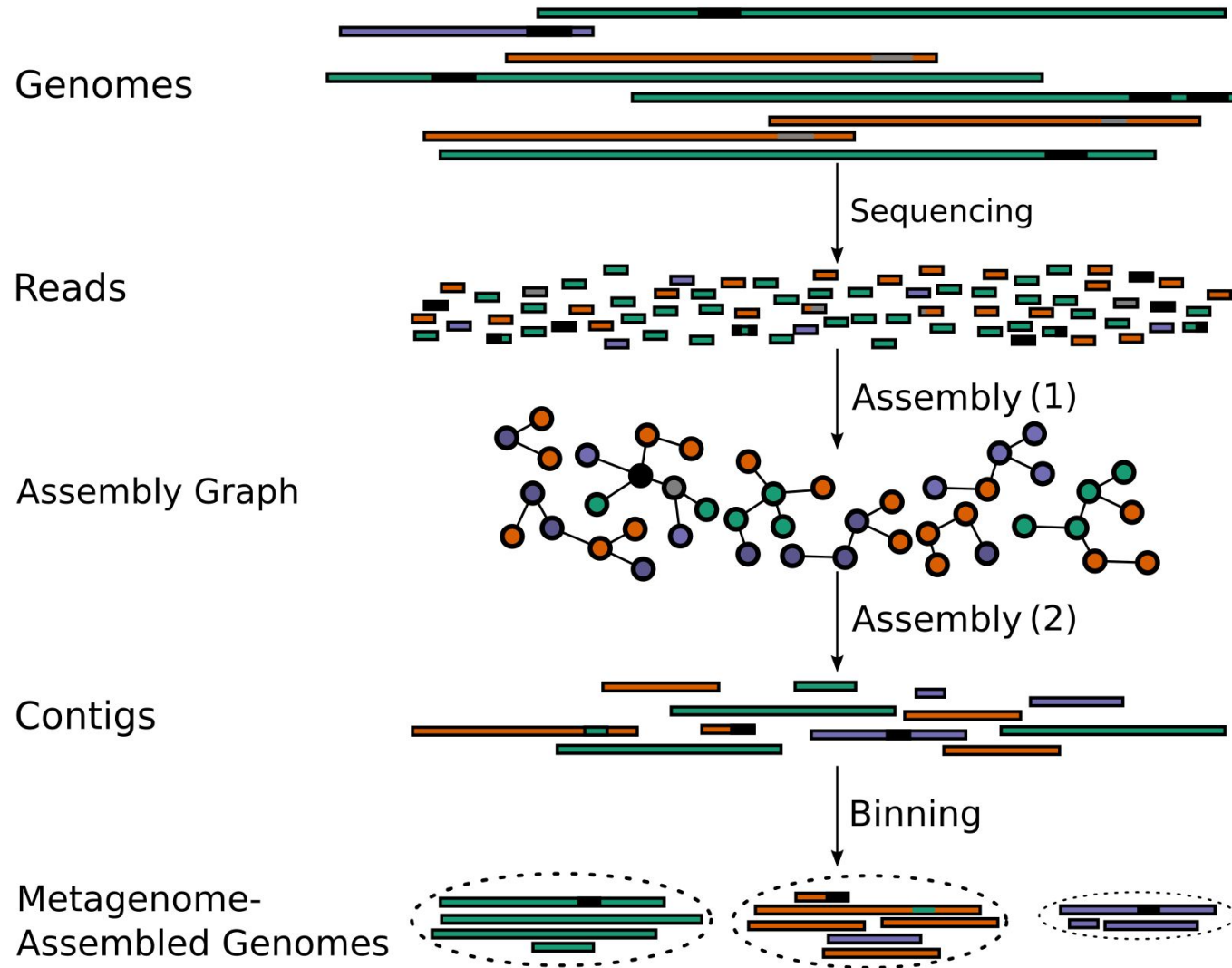


But - which genes came from which genome?

Metagenomics



Metagenomics



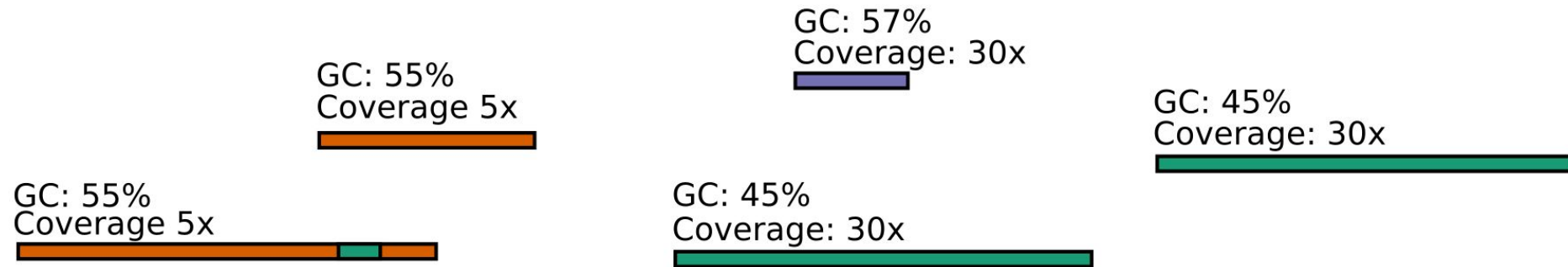
Metagenome Assembled Genome (MAG) Binning



Metagenome Assembled Genome (MAG) Binning



Metagenome Assembled Genome (MAG) Binning



Metagenome Assembled Genome (MAG) Binning

GC: 55%
Coverage 5x



GC: 55%
Coverage 5x



GC: 57%
Coverage: 30x



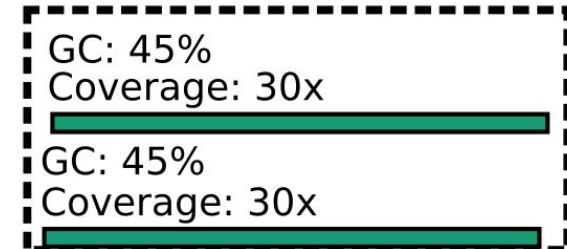
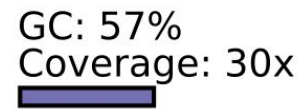
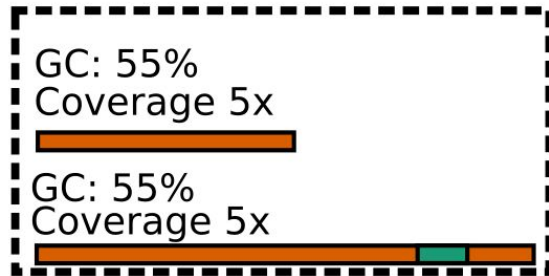
GC: 45%
Coverage: 30x



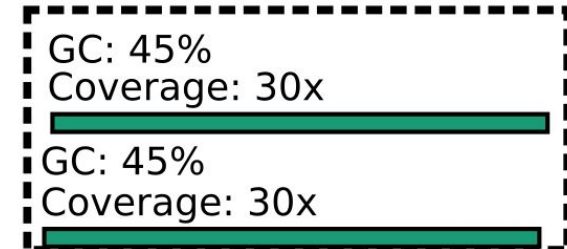
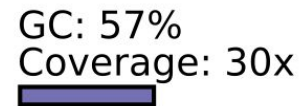
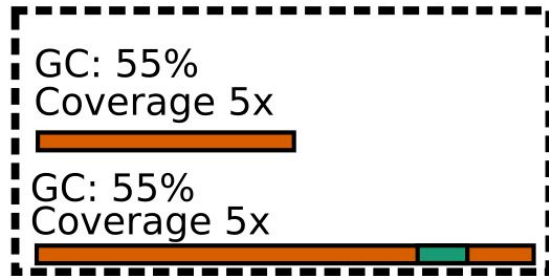
GC: 45%
Coverage: 30x



Metagenome Assembled Genome (MAG) Binning



Metagenome Assembled Genome (MAG) Binning

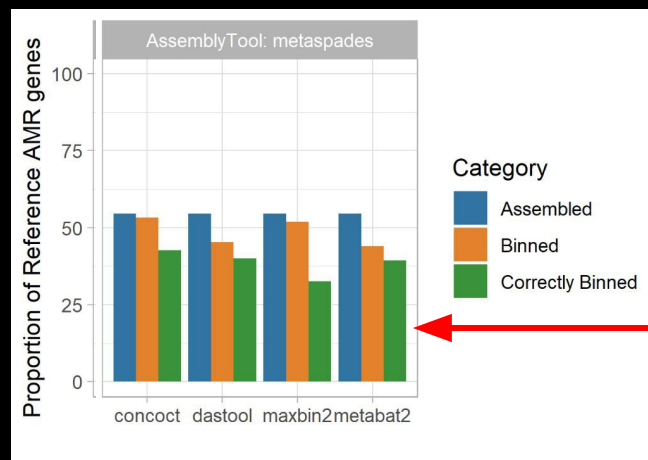


Metabat and many other MAG binning tools cluster contigs using **composition** and **coverage** data.

Dealing with copy number variation and compositional differences: **covariance** across samples

This is Great, But Good Luck...

- Resolving closely related strains with similar / the same nucleotide compositions
- Resolving repeats (even harder!)
- Doing much with rare species
- Recovering plasmids and things frequently found on plasmids



Recovery of antimicrobial resistance genes is TERRIBLE

Summary

- Assembly is important, even if we wish we didn't have to do it
- As we have seen elsewhere (and will see), the choice of algorithm is key (intuition vs. efficiency)
- Assembly of a single genome is hard enough, but techniques such as metagenomic analysis make the problem even more difficult

